



UNIVERSITÀ
POLITECNICA
DELLE MARCHE

Università Politecnica delle Marche

Facoltà di Ingegneria Informatica e dell'Automazione

Classification, Clustering & Timeseries

Analisi dei crimini nella città di Baltimora

Autori:

Matteo Sonaglioni

Enzo Cingoli

Gabriel Vladut Popa

Anno Accademico 2025–2026

Relazione redatta per il corso di Data Science

Indice

1	Crimes in Baltimora	4
1.1	Crimes in Baltimora Analysis	4
1.2	Data Preprocessing ed ETL (Extract, Transform, Load)	4
1.2.1	Fase di pulizia e allineamento iniziale	4
2	Classification	7
2.1	Preparazione dati	7
2.2	Classificazione	10
2.3	Risultati della Classificazione	11
2.4	Classificazione con SMOTEENN	14
2.5	Risultati della Classificazione con SMOTEENN	16
2.6	Classificazione con SMOTEENN Custom (Sulle classi 0,2,3)	19
2.7	Risultati della Classificazione con SMOTEENN Custom	20
2.8	Classificazione con Smote	23
2.9	Risultati della Classificazione con SMOTE	23
2.10	Conclusioni e osservazioni	25
3	Clustering	28
3.1	Preprocessing e pulizia del dataset	28
3.1.1	Rimozione delle colonne non necessarie	28
3.1.2	Gestione delle variabili temporali	29
3.1.3	Estrazione delle coordinate geografiche	29
3.1.4	Pulizia e normalizzazione delle variabili categoriche	29
3.1.5	Gestione dei valori mancanti	29
3.1.6	Dataset finale	30
3.2	Preparazione dei dati per il clustering	30
3.3	Clustering degli eventi senza filtraggio DBSCAN	31
3.3.1	Scelta del numero di cluster	31
3.3.2	Applicazione di KMeans	32
3.3.3	Visualizzazione dei cluster tramite PCA	33
3.3.4	Profilazione dei cluster	34
3.3.5	Analisi visiva dei profili di cluster tramite heatmap	35
3.3.6	Analisi tramite lift	38
3.4	Clustering degli eventi con filtraggio DBSCAN	40
3.4.1	Filtraggio preliminare con DBSCAN	40

3.4.2	Scelta del numero di cluster	42
3.4.3	Applicazione di KMeans sul dataset filtrato	43
3.4.4	Visualizzazione e interpretazione dei cluster tramite PCA . . .	44
3.4.5	Profilazione dei cluster	47
3.4.6	Analisi dei profili di cluster tramite heatmap	48
3.4.7	Analisi tramite lift	49
3.5	Confronto tra clustering con e senza filtraggio DBSCAN	51
3.5.1	Struttura e distribuzione dei cluster	52
3.5.2	Confronto tramite PCA	52
3.5.3	Heatmap vs Lift	53
3.5.4	Considerazioni conclusive	53
4	Timeseries	54
4.1	Preparazione dati	54
4.2	Analisi esplorativa del dataset	55
4.3	Addestramento del modello sulla serie originale	65
4.4	Analisi della serie differenziata	68
4.5	Addestramento del modello sulla serie differenziata	77
4.6	Miglioramento del modello SARIMAX: Feature Engineering e Varia- bili Esogene	79
4.7	Conclusioni ed osservazioni	85

1 Crimes in Baltimora

1.1 Crimes in Baltimora Analysis

Le presenti analisi si fondano sull'utilizzo del dataset "City of Baltimore Crime Data". Tale raccolta di dati offre una granularità temporale e spaziale significativa, includendo eventi criminali registrati nella città di **Baltimora (Maryland, USA)** in un intervallo esteso compreso tra il **2010 e il 2017**. Ogni record documenta in dettaglio eventi specifici, fornendo variabili essenziali quali la **data e l'ora** dell'evento, la **tipologia di reato**, la **localizzazione geografica** e l'identificazione del **distretto di polizia** di competenza. La ricchezza e la multidimensionalità di queste informazioni lo rendono un *corpus* ideale per l'applicazione di metodologie di **Analisi delle Serie Temporal**i per la previsione dell'andamento evolutivo dei crimini aggregati, il **Clustering** per l'identificazione di profili criminali dei vari distretti e la **Classification** per la categorizzazione di variabili correlate all'evento.

1.2 Data Preprocessing ed ETL (Extract, Transform, Load)

1.2.1 Fase di pulizia e allineamento iniziale

La fase di *Extract* e la pulizia iniziale sono state uniformi per tutti gli approcci di modellazione. Il processo ha avuto inizio con il caricamento del dataset in formato ".csv".

Il dataset analizzato contiene **285.807** osservazioni e **12** colonne.

La seguente tabella (Tabella 1) riassume la struttura e la descrizione delle variabili presenti nel dataset:

Tabella 1: Struttura e descrizione delle colonne del dataset Crime in Baltimora.

Colonna	Tipo	Descrizione
CrimeDate	object	Data in cui il crimine è stato registrato.
CrimeTime	object	Ora dell'evento.
CrimeCode	object	Codice identificativo del tipo di reato.
Location	object	Indirizzo o descrizione del luogo. Alcuni valori mancanti ($\approx 0.6\%$).
Description	object	Descrizione testuale del tipo di crimine (es. "ROBBERY").
Inside/Outside	object	Indica se il crimine è avvenuto all'interno o all'esterno. Alcuni valori mancanti ($\approx 1.5\%$).
Weapon	object	Tipologia di arma utilizzata. Alta percentuale di valori mancanti ($\sim 66\%$).
Post	float64	Numero del distretto di polizia (post).
District	object	Nome del distretto di polizia (es. "SOUTHERN"). Pochi valori mancanti.
Neighborhood	object	Nome del quartiere di Baltimora. Alcuni valori mancanti ($\approx 0.6\%$).
Location 1	object	Coordinate geografiche (latitudine, longitudine) in formato stringa.
Total Incidents	int64	Numero totale di incidenti registrati (spesso pari a 1).

Il dataset si presenta **abbastanza completo**, con una percentuale di valori nulli limitata in quasi tutte le colonne ad eccezione di **Weapon**, il che impone particolare attenzione durante la fase di pulizia e nella scelta del **target** per la classificazione. I dati sono **prevalentemente categoriali e temporali**, rendendoli particolarmente adatti per l'applicazione di **modelli di forecasting** sul numero di incidenti nel tempo.

1. Gestione dei Valori Mancanti

- Le colonne **Location**, **Inside/Outside**, **Weapon**, **District** e **Neighborhood** presentano una certa quantità di valori nulli.

- Si è scelto di sostituire tali valori con una categoria "Unknown" per preservare il numero di osservazioni e non introdurre distorsioni, ad eccezione dei casi in cui la colonna era usata come target (e.g., *Weapon* nella classificazione).

2. Creazione della Colonna Temporale Unificata

- Le colonne `CrimeDate` e `CrimeTime` sono state unite in un'unica colonna `Datetime` con la seguente operazione:

```
1 crime_data['Datetime'] = pd.to_datetime(
2     crime_data['CrimeDate'] + ' ' + crime_data['CrimeTime'],
3     errors='coerce')
```

- Questa nuova variabile temporale è cruciale per:
 - ordinare gli eventi nel tempo (`sort_values('Datetime')`);
 - effettuare analisi stagionali o giornaliere;
 - eseguire *resampling* (su base giornaliera o settimanale) per l'analisi delle serie storiche.

3. Verifica e Coerenza dei Dati Temporali

- Infine, i dati sono stati ordinati e l'indice è stato resettato per assicurare la coerenza temporale in preparazione per l'analisi sequenziale:

```
1 crime_data = (
2     crime_data.sort_values('Datetime')
3     .reset_index(drop=True))
```

Dopo questa fase generale, il flusso di *Transformation e Load* si è differenziato per soddisfare i requisiti algoritmici specifici di ciascuna analisi.

2 Classification

L'obiettivo dell'analisi di **Classificazione** consiste nel prevedere la tipologia di arma impiegata durante ciascun evento criminale, rappresentata dalla variabile **Weapon**. Il modello sviluppato ha il compito di assegnare ogni osservazione a una delle seguenti classi: *HANDS* (classe 1), *KNIFE* (classe 2), *FIREARM* (classe 0) o *OTHER* (classe 3).

Per la realizzazione del processo di analisi sono state impiegate le principali librerie del linguaggio **Python**, tra cui **pandas**, **matplotlib**, **seaborn**, **statsmodels**, **numpy**, **imblearn** e **scikit-learn**. Tali strumenti hanno consentito di eseguire una **pulizia approfondita dei dati**, un'accurata **analisi esplorativa** e la successiva **modellazione predittiva**, finalizzata alla costruzione e valutazione di modelli di classificazione.

2.1 Preparazione dati

Il passaggio critico è il filtraggio iniziale del dataset, necessario perché i valori della categoria **Unknown** (che è dominante, con circa il 66%) è stata **rimossa dal training set** per non falsare l'addestramento, riducendo significativamente la dimensione del dataset finale di classificazione.

Data la natura del task e la varietà delle categorie è stato adottato il **Label Encoding** per convertire le variabili testuali in formato numerico discreto.

```
1 categorical_cols = ['CrimeCode', 'Location', 'Description',  
2                   'Weapon', 'District', 'Neighborhood', 'DayOfWeek']  
3  
4 encoders = {}  
5  
6 for col in categorical_cols:  
7     le = LabelEncoder()  
8     crime_data[f'{col}_encoded'] = le.fit_transform(crime_data[col])  
9     encoders[col] = le
```

Inoltre, è stata eseguita una **selezione preliminare delle feature** (non strettamente necessaria ma utile per l'interpretabilità) eliminando quelle con un'elevata correlazione reciproca (coefficiente di correlazione > 0.70) per prevenire problemi di multicollinearità nel modello finale. Come si può osservare dalla **Matrice di correlazione** (Figura 1), c'è una coppia di feature fortemente correlate: **CrimeCode_encoded** e **Description_encoded**, con coefficiente di correlazione uguale

a circa **0.729**. Si decide quindi di **eliminare** la feature **CrimeCode_encoded**, al fine di non influenzare la classificazione finale.

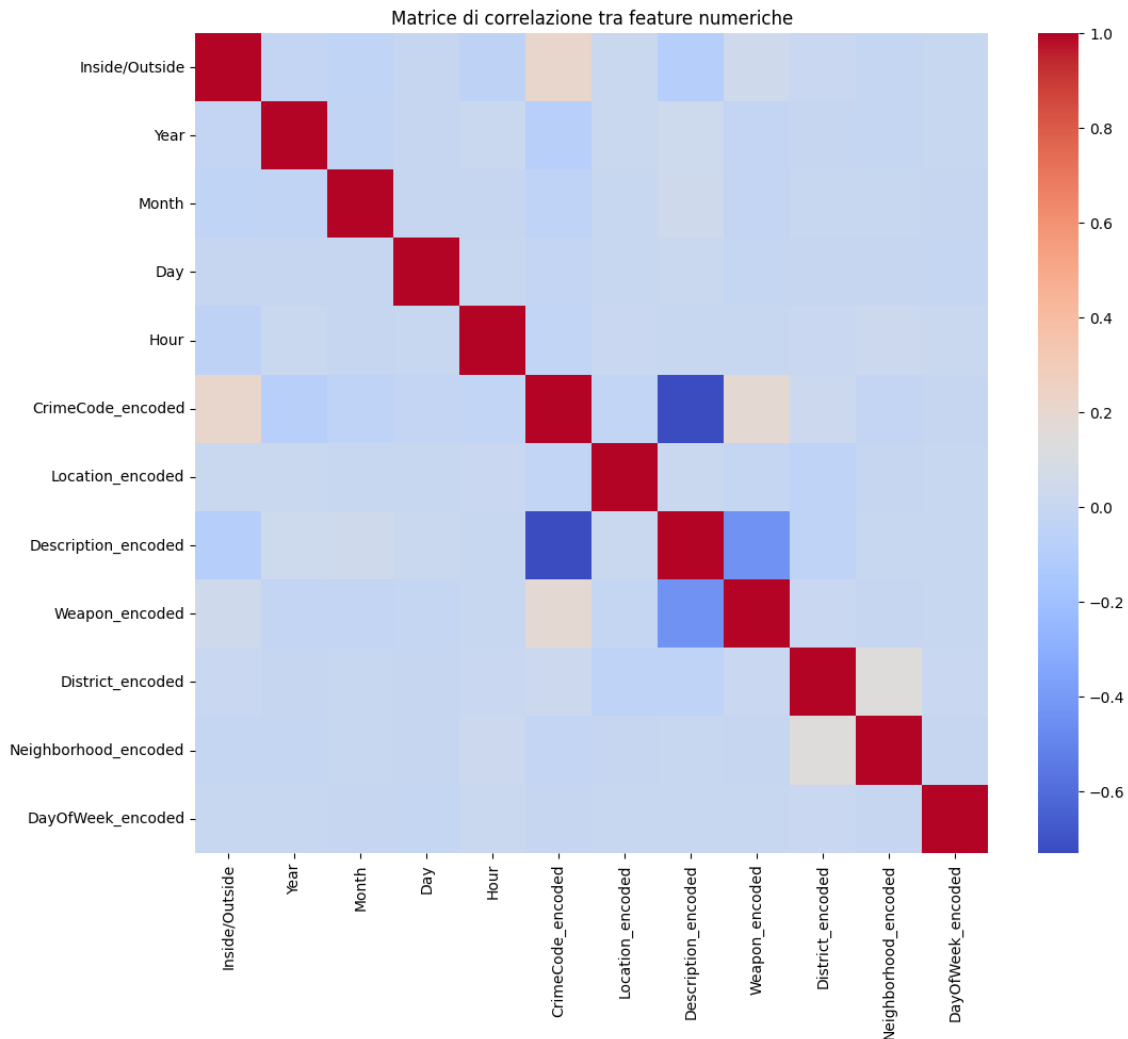


Figura 1: Matrice di correlazione tra feature numeriche

Si procede con la suddivisione del dataset finale in training (80%) e test (20%).

```

1 # Eliminazione delle feature fortemente correlate
2 crime_data_weapon = crime_data_weapon.drop(columns=['CrimeCode_encoded
  '])
3
4 scaler = MinMaxScaler()
5 cols_to_scale = crime_data_weapon.columns.drop('Weapon_encoded')
6
7 crime_data_weapon_normalized = crime_data_weapon.copy()
8 crime_data_weapon_normalized[cols_to_scale] = scaler.fit_transform(
  crime_data_weapon[cols_to_scale])

```



```

9
10 X = crime_data_weapon.drop('Weapon_encoded', axis=1)
11 y = crime_data_weapon['Weapon_encoded']
12
13 # Split del dataset in train e test
14 X_train, X_test, y_train, y_test = train_test_split(
15     X, y, test_size=0.2, random_state=42, stratify=y
16 )

```

Osservando il numero di elementi per ogni classe di **Weapon_encoded** e la loro distribuzione, si ottiene il seguente output:

```

1 --- Distribuzione delle classi nel dataset di training ---
2 Weapon_encoded
3 1 55.472387
4 0 18.494373
5 3 15.480167
6 2 10.553074
7 Name: proportion, dtype: float64
8
9 --- Numero di elementi per ogni classe nel dataset di training ---
10 Weapon_encoded
11 1 41353
12 0 13787
13 3 11540
14 2 7867
15 Name: count, dtype: int64
16
17 --- Distribuzione delle classi nel dataset di test ---
18 Weapon_encoded
19 1 55.470301
20 0 18.495466
21 3 15.479959
22 2 10.554274
23 Name: proportion, dtype: float64
24
25 --- Numero di elementi per ogni classe nel dataset di test ---
26 Weapon_encoded
27 1 10338
28 0 3447
29 3 2885
30 2 1967
31 Name: count, dtype: int64

```

Listing 1: Distribuzione delle classi di `Weapon_encoded` nei dataset di training e test.

L'analisi evidenzia una predominanza della **Classe 1 (*HANDS*)**, che rappresenta circa il 55% delle osservazioni sia nel dataset di training sia in quello di test. Le altre classi risultano significativamente meno rappresentate, indicando uno **sbilanciamento** nella distribuzione delle categorie. Si osserva, inoltre, una buona coerenza tra training e test set, in quanto le proporzioni relative delle classi sono sostanzialmente simili, garantendo che il campionamento preservi la struttura originale dei dati.

Tale sbilanciamento **potrebbe influenzare negativamente** le prestazioni di modelli predittivi standard, poiché il modello potrebbe tendere a privilegiare la classe più frequente.

2.2 Classificazione

Per affrontare il task di Classificazione è stato impiegato un *ensemble* diversificato di algoritmi di *Machine Learning*. La scelta di includere modelli sia semplici che complessi mira a valutare quale approccio garantisca il **miglior equilibrio** tra performance e capacità di generalizzazione sui dati.

Nel seguente listato vengono descritti i classificatori utilizzati per la fase di training.

```
1 classifiers = [  
2     ("DecisionTreeClassifier", DecisionTreeClassifier(random_state=42)  
3         , {"max_depth": [10,13,15,18, 20, 23]}),  
4     ("RandomForestClassifier", RandomForestClassifier(random_state=42)  
5         , {"n_estimators": [5,10,15, 25, 30], "max_depth":  
6             [10,13,15,18, 20, 23]}),  
7     ("AdaBoostClassifier", AdaBoostClassifier(random_state=42), {"  
8         n_estimators": [5,10,15, 25, 30]}),  
9     ("GradientBoostingClassifier", GradientBoostingClassifier(  
10         random_state=42), {"n_estimators": [5,15,25, 25, 30]}),  
11     ("NeuralNetwork", Pipeline([  
12         ('scaler', StandardScaler()),  
13         ('clf', MLPClassifier(  
14             hidden_layer_sizes=(64, 32, 16),  
15             activation='relu',  
16             solver='adam',
```

```

12         max_iter=100,
13         random_state=42
14     ))
15 ], {"clf__hidden_layer_sizes": [(64,32,16),(32,16,8)], "
    clf__max_iter": [300,400]})
16 ]

```

Verrà utilizzato un modello di **GridSearchCV** per trovare automaticamente la combinazione ottimale di iperparametri, attraverso una **cross-validation**. Verranno testati **6 alberi decisionali diversi**, ognuno con una profondità diversa ed usando **4-fold cross validation** per stimare l'Accuracy media di ogni figurazione.

2.3 Risultati della Classificazione

Di seguito, sono riportati i risultati della valutazione, utilizzando metriche di classificazione di **Accuracy**, **Precision**, **Recall**, **F1-Score** (in particolare il *macro avg* che valuta l'equilibrio tra classi) e l'**Area Under the Curve (AUC)**.

Tabella 2: Metriche di valutazione dei modelli di classificazione sul dataset.

Modello	Accuracy	Precision (Macro Avg)	Recall (Macro Avg)	F1-Score (Macro Avg)
Random Forest	0.796	0.655	0.672	0.652
Gradient Boosting	0.794	0.627	0.655	0.631
Ada Boost	0.783	0.628	0.617	0.617
Decision Tree	0.761	0.589	0.598	0.593
Rete Neurale (MLP)	0.755	0.559	0.589	0.569

Come si può osservare dalla Tabella 2, il **Random Forest** ha ottenuto le performance migliori in tutte le metriche aggregate. Seguono **GradientBoosting**, **AdaBoostClassifier** e **DecisionTree**, che si comportano in modo molto simile, con prestazioni comunque buone. Il modello **NeuralNetwork** ha prestazioni inferiori, con F1-Score particolarmente basso, indicando che fa molta fatica a bilanciare Precision e Recall.

Confronto Matrici di Confusione

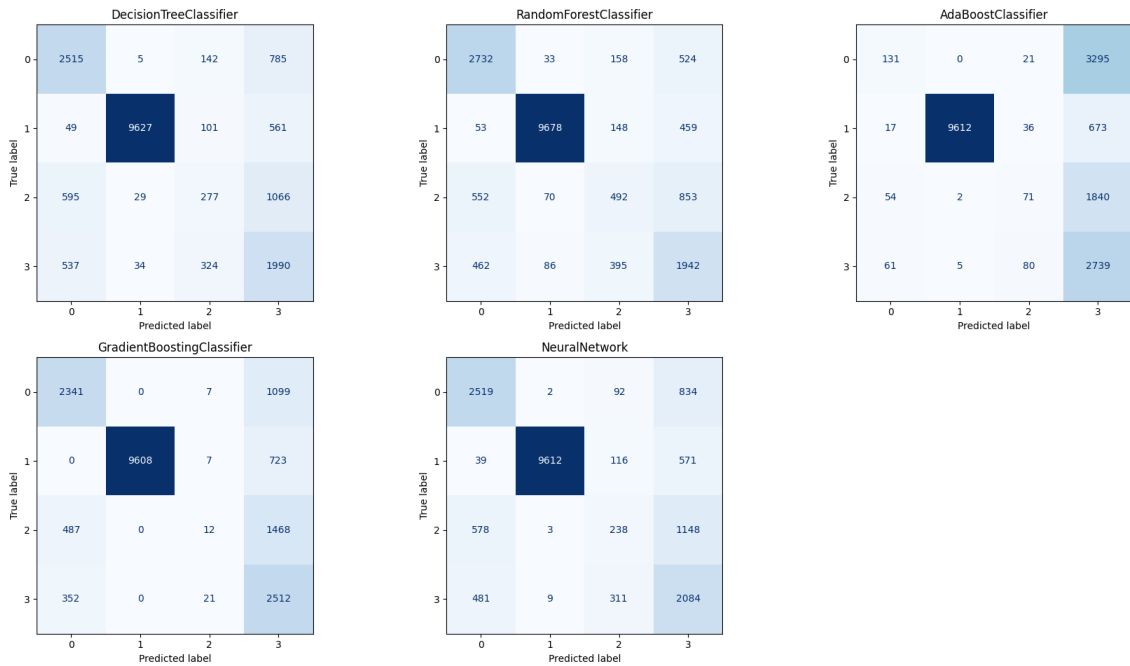


Figura 2: Confronto matrici di Confusione di ciascun modello

Analizzando le Matrici di confusione (Figura 2), emergono le sfide più interessanti: il problema principale è il **forte squilibrio delle classi**. Questo si può capire anche dal confronto tra Accuracy e F1-Score, dove l'Accuracy è alta, mentre l'F1-Score è più basso. Questa discrepanza suggerisce che l'Accuracy è "gonfiata" dalla capacità del modello di **indovinare correttamente la classe maggioritaria**. Infatti, osservando le matrici di confusione, si può notare come **tutti i modelli sono eccellenti nel predire la Classe 1**. Le **Classi 0 e 2** vengono spesso confuse ed erroneamente classificate come **Classe 3**.

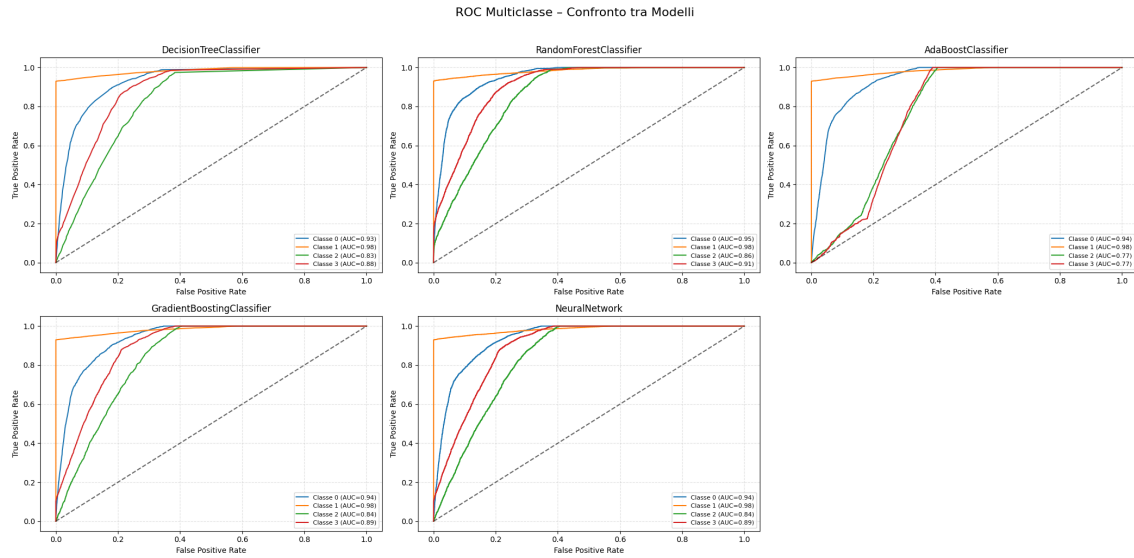


Figura 3: Confronto della curva ROC di ogni modello

Le curve ROC (Figura 3), mostrano l'abilità del modello di distinguere tra le classi.

- La **performance è buona**: le aree sotto la curva (AUC) sono molto alte per quasi tutti i modelli. Un AUC alto significa che il modello assegna probabilità "corrette" (ad esempio un campione di Classe 0 ha una probabilità più alta di essere Classe 0 che di essere Classe 3). Tuttavia, quando si passa dalle probabilità alla classificazione finale, il modello "sceglie" la classe che massimizza la probabilità. A causa dello squilibrio, il modello si "sbilancia" preferendo la Classe 3 anche quando la probabilità per la Classe 0 o 2 è discreta. In breve, i modelli **hanno distinguere le classi** (alto AUC), ma **fanno la scelta sbagliata** al momento della classificazione finale a causa dello squilibrio dei dati.

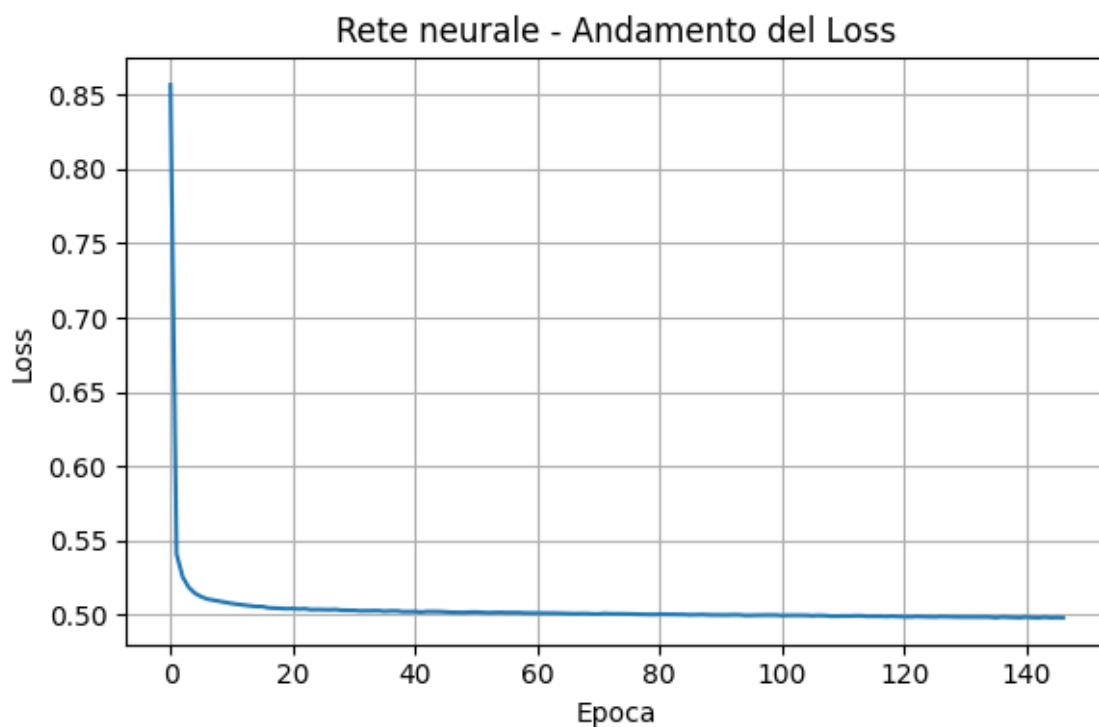


Figura 4: Andamento della Loss della rete neurale

Per quanto riguarda l'andamento della Loss della **Rete Neurale** (Figura 4), si può notare che:

- **Convergenza rapida:** l'errore scende verticalmente nelle prime 10-20 epoche, per poi stabilizzarsi intorno ad un valore di **0.50**.
- **Stabilità:** la curva si appiattisce e rimane bassa, indicando che il modello ha trovato una soluzione stabile e non sta "oscillando" o peggiorando nelle epoche successive.

Inoltre, tutti i modelli sono stati **estremamente veloci** (poche decine di secondi al massimo) nell'addestramento. L'unica eccezione è la **Rete Neurale** che, pur essendo un modello avanzato, ha richiesto **tempi di addestramento significativamente maggiori (fino a 5 minuti)**.

In conclusione, il vero problema del dataset è quindi l'**impatto dello squilibrio delle classi**. L'Accuracy è alta, ma le prestazioni sulle classi minoritarie sono scarse.

2.4 Classificazione con SMOTEENN

Dal momento che le predizioni risultano discrete per la classe maggioritaria (*HANDS*), ma piuttosto povere per le restanti classi, si tenta l'implementazione di una tecnica

ibrida **Upsampling + Downsampling**: si intende aumentare artificialmente il numero di campioni presenti nel dataset di training, inerenti alle classi 0 ('FIREARM'), 2 ('KNIFE') e 3 ('OTHER') ed effettuarne poi una pulizia.

Si utilizzerà la tecnica **SMOTE+ENN** sul training set per creare "nuovi" campioni delle classi minoritarie (Tabella 3). La prima fase, di SMOTE, aumenta la mole dei dati delle classi minoritarie fino al livello (100%) della classe maggioritaria. La seconda fase di ENN (Edited Nearest Neighbors) elimina i dati in base ai propri vicini (se i neighbors sono in maggioranza di calsse differene, il dato viene eliminato), al fine di rimuovere rumore e outliers.

Usando una **Pipeline** viene assicurato che il ricampionamento avvenga solo sul set di addestramento all'interno di ogni singolo fold della cross-validation, evitando di "contaminare" i dati di validazione.

Tabella 3: Media del numero di campioni per classe prima e dopo l'applicazione di SMOTEENN (media sui fold di cross-validation).

Classe	Originale (Train)	SMOTEENN
FIREARM	10 340.25	19 445.25
HANDS	31 014.75	8 246.00
KNIFE	5 900.25	21 357.50
OTHER	8 655.00	18 814.50

```

1 classifiers_smoteenn = [
2     ("DecisionTreeClassifier",
3     ImbPipeline([
4         ('resampling', SMOTEENN(random_state=42)),
5         ('model', DecisionTreeClassifier(random_state=42))
6     ]),
7     {"model__max_depth": [10, 13, 15, 18, 20, 23]}),
8     ("RandomForestClassifier",
9     ImbPipeline([
10        ('resampling', SMOTEENN(random_state=42)),
11        ('model', RandomForestClassifier(random_state=42))
12    ]),
13    {"model__n_estimators": [5, 10, 15, 25, 30], "model__max_depth":
14        [10, 13, 15, 18, 20, 23]}),
15    ("AdaBoostClassifier",
16    ImbPipeline([
17        ('resampling', SMOTEENN(random_state=42)),
18        ('model', AdaBoostClassifier(random_state=42))

```

```

18     ]),
19     {"model__n_estimators": [5, 10, 15, 25, 30]}),
20     ("GradientBoostingClassifier",
21     ImbPipeline([
22         ('resampling', SMOTEENN(random_state=42)),
23         ('model', GradientBoostingClassifier(random_state=42))
24     ]),
25     {"model__n_estimators": [5, 15, 25, 25, 30]}),
26     ("NeuralNetwork",
27     ImbPipeline([
28         ('resampling', SMOTEENN(random_state=42)),
29         ('model', Pipeline([
30             ('scaler', StandardScaler()),
31             ('clf', MLPClassifier(
32                 hidden_layer_sizes=(64, 32, 16),
33                 activation='relu',
34                 solver='adam',
35                 max_iter=100,
36                 random_state=42
37             ))
38         ]))
39     ]),
40     {"model__clf__hidden_layer_sizes": [(64, 32, 16), (32, 16, 8)],
41      "model__clf__max_iter": [300, 400]})
42 ]

```

Come prima, verrà utilizzato il modello **GridSearchCV**, con gli stessi iperparametri.

2.5 Risultati della Classificazione con SMOTEENN

Di seguito, si riportano i risultati della seconda valutazione, utilizzando le stesse metriche di classificazione e si osserverà se la tecnica ibrida di Upsampling+Downsampling utilizzata ha effettivamente **migliorato** i modelli.

Tabella 4: Confronto delle metriche di performance dei modelli di classificazione con SMOTEENN.

Modello	Accuracy	Precision (Macro)	Recall (Macro)	F1-Score (Macro)
Decision Tree Classifier	0.760	0.613	0.623	0.616
Random Forest Classifier	0.779	0.638	0.651	0.643
AdaBoost Classifier	0.773	0.537	0.616	0.558
Gradient Boosting Classifier	0.763	0.621	0.628	0.622
Rete Neurale (MLP)	0.761	0.612	0.625	0.616

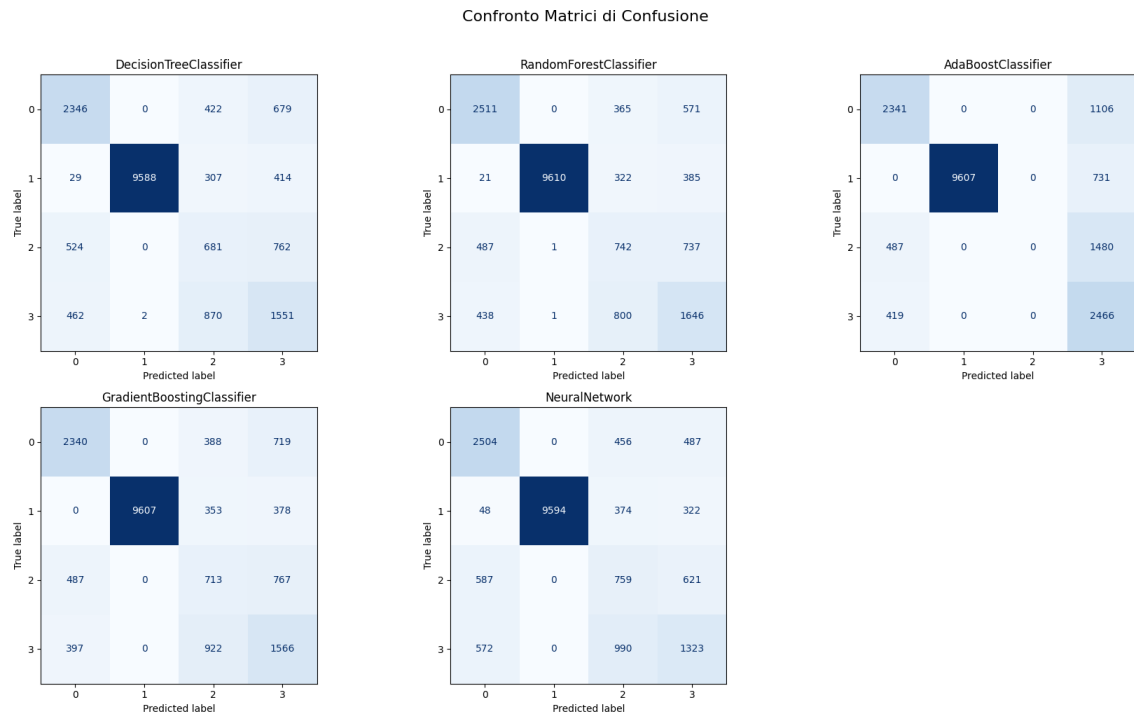


Figura 5: Confronto Matrici di Confusione con SMOTEENN

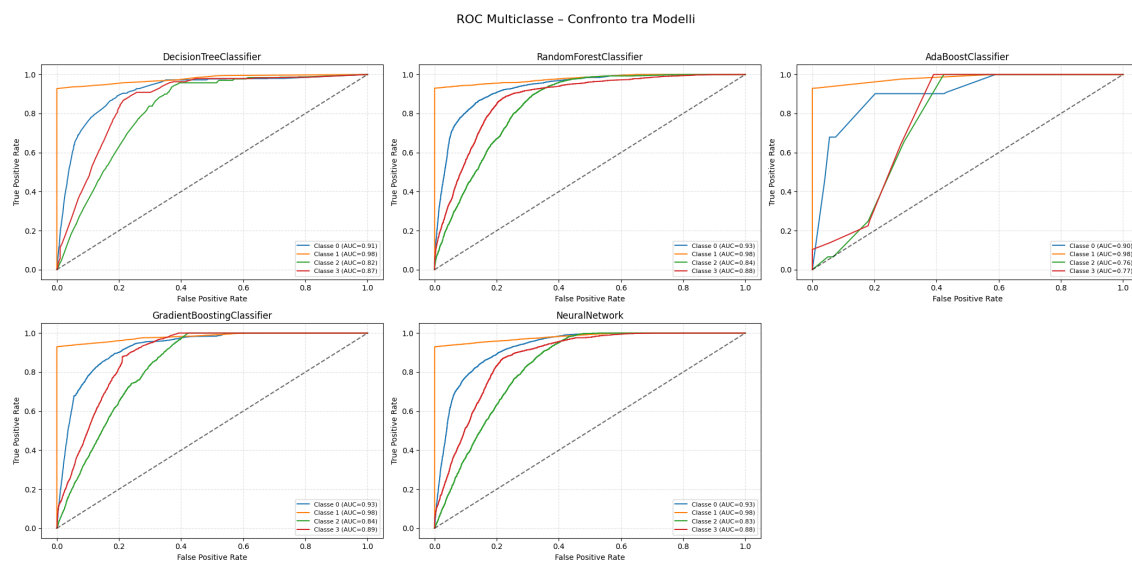


Figura 6: Confronto ROC con SMOTEENN

Osservando le **Curve ROC** (Figura 6), si può notare che per quasi tutti i modelli, i valori di **AUC** sono **scesi** rispetto al primo esperimento. Questo significa che i dati sintetici di SMOTEENN hanno reso i confini tra le classi più "confusi", rendendo più difficile per i modelli distinguere le classi.

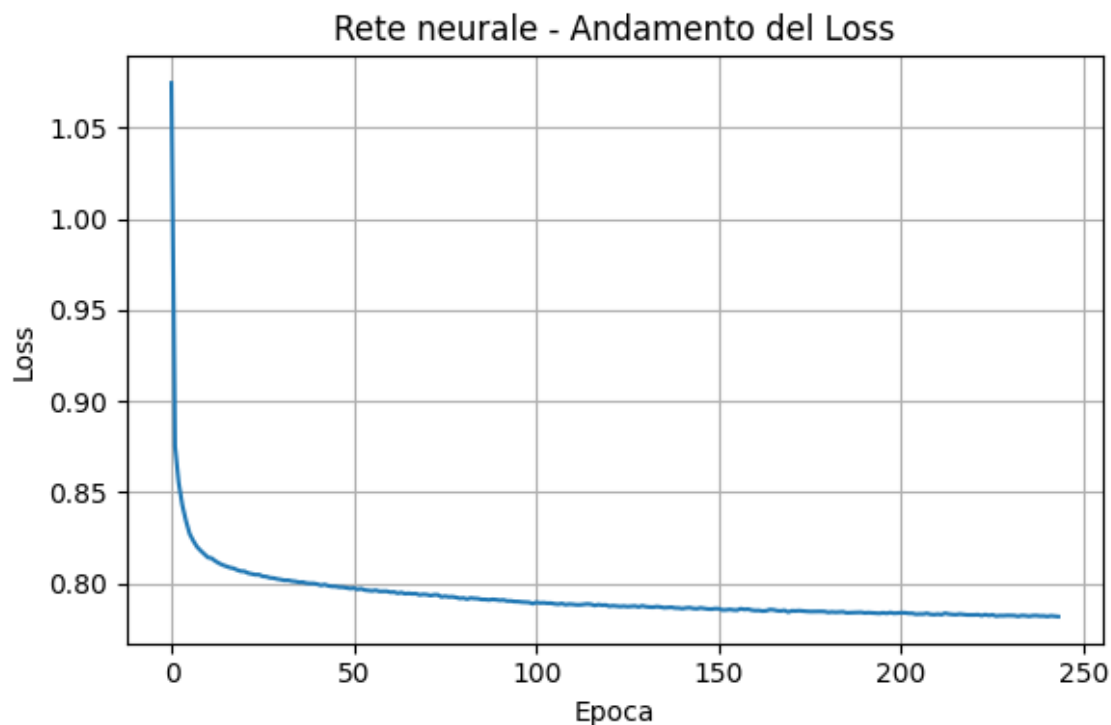


Figura 7: Andamento Loss con SMOTEENN

Anche la **Loss della rete neurale** (Figura 7) ora parte molto più alta e si

stabilizza più in alto (circa **0.78** contro **0.50**). Questo conferma che il nuovo dataset ricampionato è più rumoroso e più difficile da imparare per il modello.

2.6 Classificazione con SMOTEENN Custom (Sulle classi 0,2,3)

Abbiamo deciso di effettuare un altro tentativo di classificazione con una funzione di SMOTEENN Custom, la quale non intaccasse il numero di istanze della classe maggioritaria (come avveniva nella sezione precedente). La mole di dati ottenuti in questa fase è presente nella Tabella 5.

Come si voleva ottenere, la classe maggioritaria non è stata intaccata, mentre le classi minoritarie hanno riaggiunto un livello di dati equiparabile a quello della classe HANDS.

Tabella 5: Media del numero di campioni per classe prima e dopo l'applicazione di SMOTEENN Custom (media sui fold di cross-validation).

Classe	Originale (Train)	SMOTEENN Custom
FIREARM	10 340.25	26 863.00
HANDS	31 014.75	31 014.75
KNIFE	5 900.25	27 238.00
OTHER	8 655.00	25 909.00

Nel seguente codice è riportata la sequenza necessaria per la creazione del nuovo metodo di Upsampling con parametri custom.

```

1      # 1. Configurazione SMOTE (Upsampling Auto)
2  smote_config = SMOTE(
3      sampling_strategy='auto',
4      k_neighbors=5,
5      random_state=42
6  )
7
8  # 2. Configurazione ENN (Downsampling / Pulizia)
9  # sampling_strategy='not majority' pulisce tutto tranne la classe
    maggioritaria.
10 enn_config = EditedNearestNeighbours(
11     sampling_strategy=[0, 2, 3],
12     n_neighbors=3,
13     kind_sel='mode'
14 )

```

```

15
16 # 3. Combinazione SMOTE + ENN
17 smote_enn_custom = SMOTEENN(
18     smote=smote_config,
19     enn=enn_config,
20     random_state=42
21 )

```

Successivamente, la funzione *smote_enn_custom* verrà richiamata all'interno della pipeline di GridSearch.

2.7 Risultati della Classificazione con SMOTEENN Custom

All'interno della Tabella 6 sono riportate le metriche di valutazione ottenute in seguito alla classificazione con SMOTEENN Custom.

Tabella 6: Confronto delle metriche di performance dei modelli di classificazione con SMOTEENN Custom.

Modello	Accuracy	Precision (Macro)	Recall (Macro)	F1-Score (Macro)
Decision Tree Classifier	0.767	0.617	0.629	0.622
Random Forest Classifier	0.784	0.640	0.654	0.646
AdaBoost Classifier	0.761	0.527	0.596	0.546
Gradient Boosting Classifier	0.766	0.624	0.632	0.626
Neural Network (MLP)	0.769	0.618	0.632	0.624

Di seguito sono riportate le matrici di confusione (Figura 8), andamento delle curve ROC (Figura 9) e loss della rete neurale (Figura 10).

Confronto Matrici di Confusione

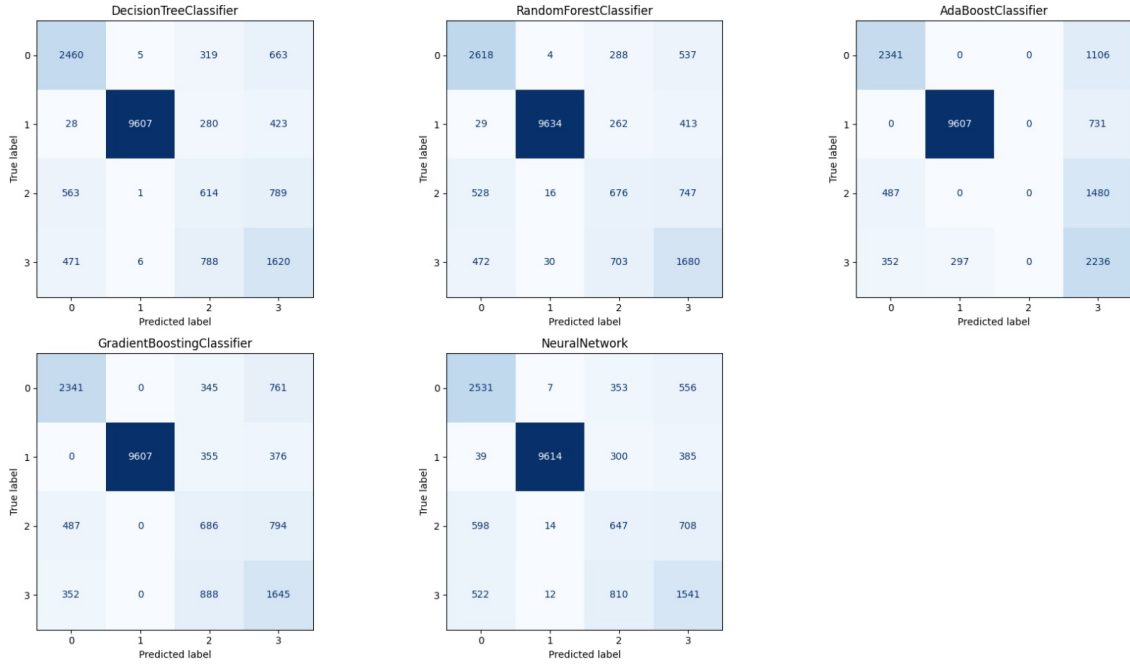


Figura 8: Confronto Matrici di Confusione con SMOTEENN Custom

ROC Multiclasse - Confronto tra Modelli

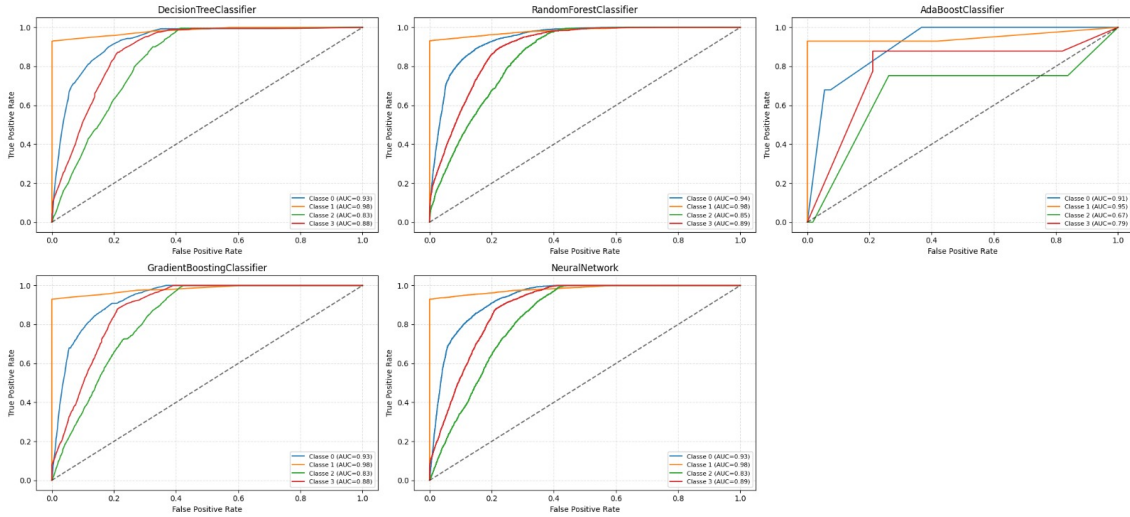


Figura 9: Confronto ROC con SMOTEENN

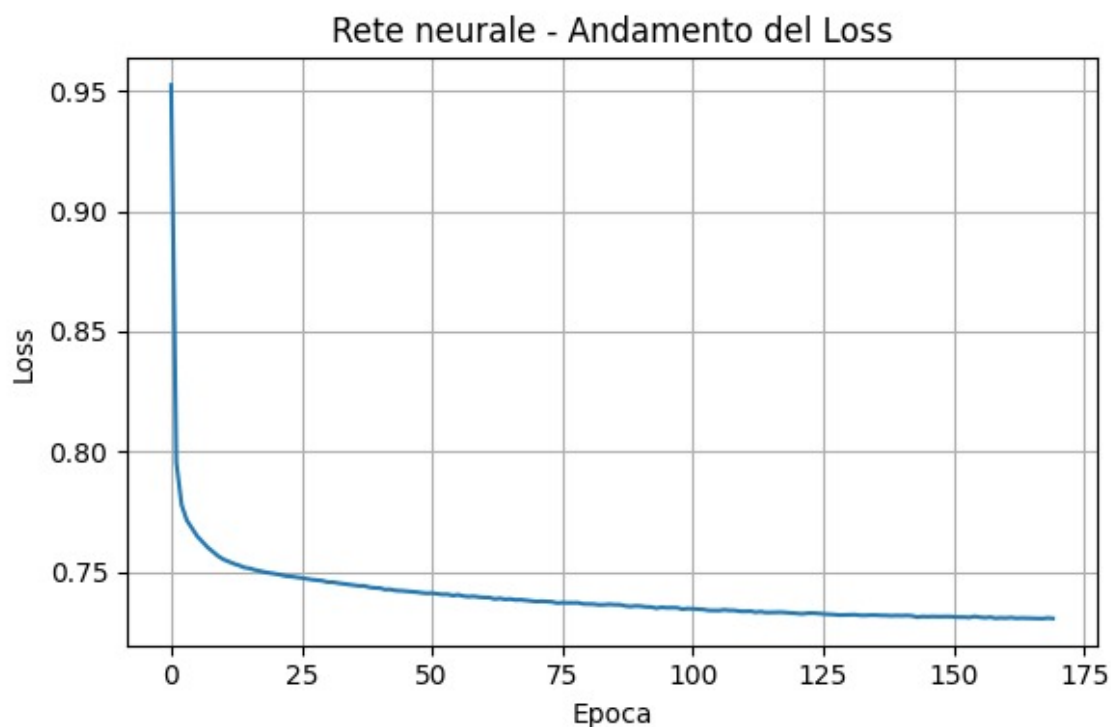


Figura 10: Andamento Loss con SMOTEENN Custom

Dai precedenti risultati emerge che:

- **Random Forest Classifier** è il modello con le migliori performance complessive:
 - **Accuracy** più alta (0.784);
 - **Precision, Recall e F1-score macro** leggermente superiori agli altri modelli;
- **Neural Network (MLP)** e **Decision Tree** mostrano performance simili:
 - Accuracy intorno a 0.767–0.769;
 - F1-score macro intorno a 0.622–0.624;
- **AdaBoost Classifier** ha la precision più bassa (0.527) rispetto agli altri modelli, pur avendo un recall relativamente più alto (0.596):
 - Tende a classificare correttamente la classe maggioritaria, ma fa più errori sulla classe 2;
 - Questo influenza l’F1-score macro, risultando inferiore rispetto agli altri modelli.

- **Gradient Boosting Classifier** ha metriche intermedie:
 - Accuracy simile al Decision Tree, ma F1-score leggermente migliore;

2.8 Classificazione con Smote

Come ultimo tentativo di migliorare la qualità delle predizioni sulla feature WEAPON, si utilizzerà la funzione SMOTE per l'upsampling, con una percentuale di aumento dei dati pari al 50% di quella maggioritaria.

Si è deciso di eliminare la parte di ENN (Downsampling) per evitare di compromettere la distribuzione di dati ottenuta nella fase di SMOTE.

Il bilanciamento delle classi dopo l'upsampling è riportato nella Tabella 7.

Tabella 7: Media del numero di campioni per classe prima e dopo l'applicazione di SMOTE (50%) sui fold di cross-validation.

Classe	Originale (Train)	SMOTE
FIREARM	10 340.25	15 507.00
HANDS	31 014.75	31 014.75
KNIFE	5 900.25	15 507.00
OTHER	8 655.00	15 507.00

2.9 Risultati della Classificazione con SMOTE

Di seguito (Tabella 8) sono presentati i valori delle metriche ottenuti tramite classificazione sui dati aumentati tramite SMOTE.

Tabella 8: Confronto delle metriche di performance dei modelli di classificazione con SMOTE.

Modello	Accuracy	Precision (Macro)	Recall (Macro)	F1-Score (Macro)
Decision Tree Classifier	0.772	0.618	0.630	0.619
Random Forest Classifier	0.792	0.650	0.664	0.655
AdaBoost Classifier	0.777	0.542	0.622	0.562
Gradient Boosting Classifier	0.774	0.628	0.634	0.620
Neural Network (MLP)	0.770	0.609	0.624	0.611

Dalla Tabella 8 si osserva che:

- **Random Forest Classifier** mostra le migliori performance complessive, con l'accuracy più alta (0.792) e macro-metriche superiori rispetto agli altri modelli. Questo indica che gli ensemble di alberi gestiscono in modo stabile le classi bilanciate tramite SMOTE.

- **Decision Tree** e **Neural Network (MLP)** riportano performance simili, con accuracy intorno a 0.770–0.772 e F1-score macro circa 0.619–0.611. Ciò suggerisce che anche modelli più semplici o più complessi riescono a bilanciare bene le classi minoritarie grazie all’oversampling.
- **AdaBoost Classifier** ha la precision più bassa (0.542) ma un recall relativamente più alto (0.622), evidenziando una tendenza a classificare correttamente alcune classi minoritarie a scapito di errori su altre classi.
- **Gradient Boosting Classifier** si colloca in una posizione intermedia, mostrando metriche bilanciate ma leggermente inferiori alla Random Forest.

Di seguito sono riportate le matrici di confusione (Figura 11), andamento delle curve ROC (Figura 12) e loss della rete neurale (Figura 13).

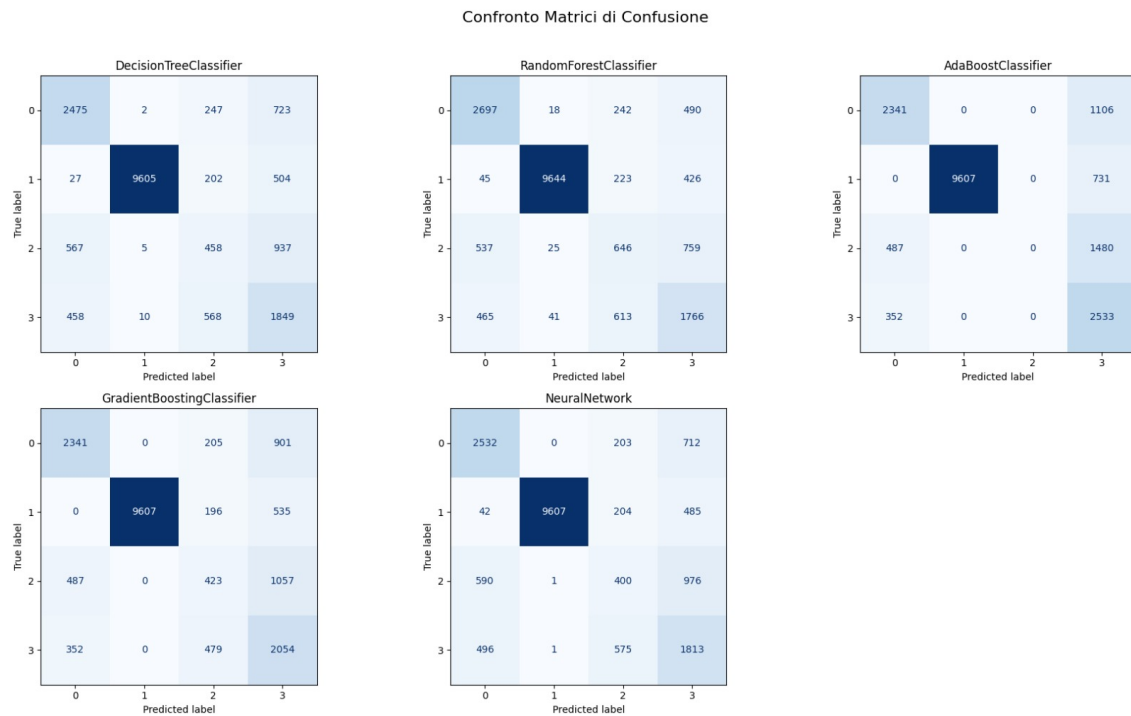


Figura 11: Confronto Matrici di Confusione con SMOTE

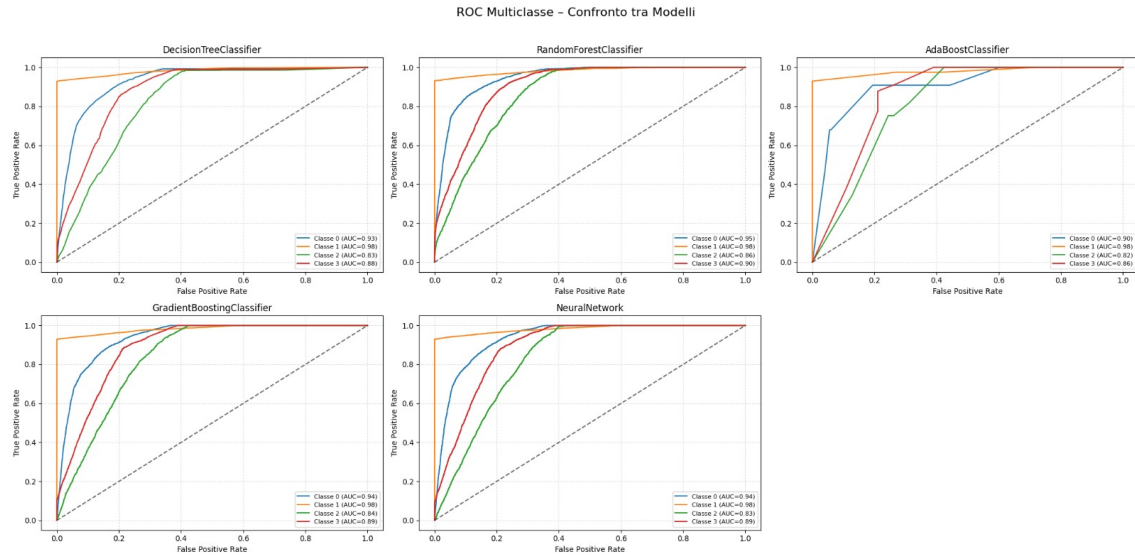


Figura 12: Confronto ROC con SMOTE

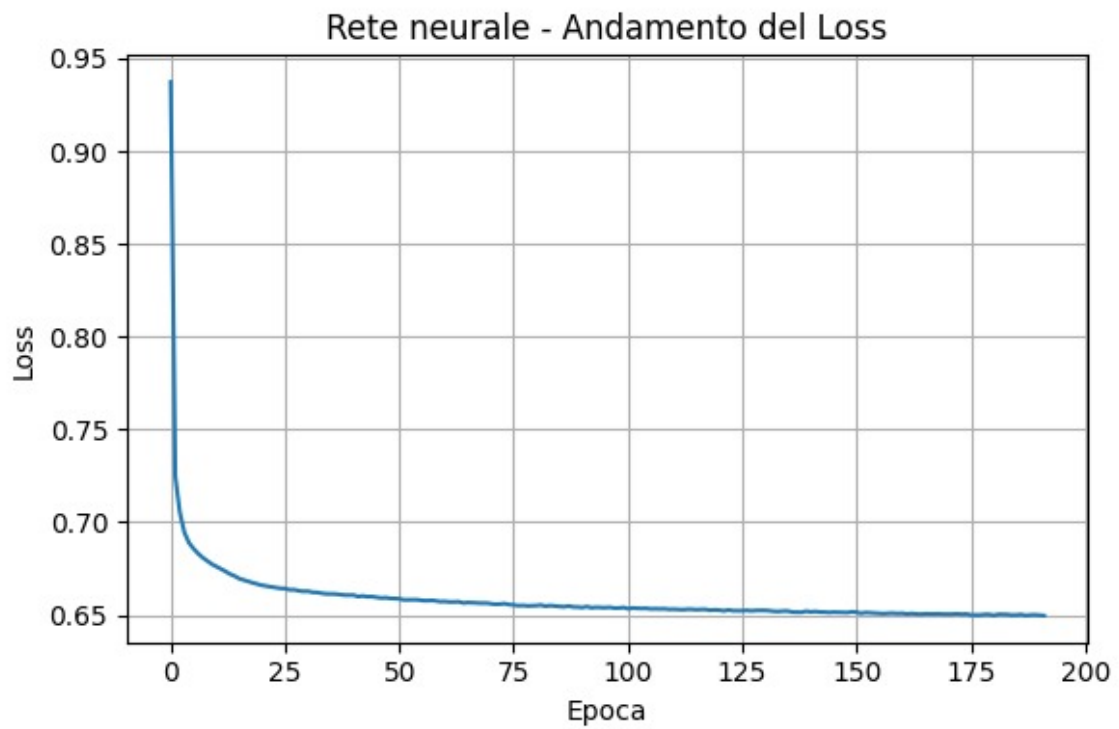


Figura 13: Andamento Loss con SMOTE

2.10 Conclusioni e osservazioni

Di seguito, nella Tabella 9, sono stati riportati i risultati ottenuti tramite i vari approcci di Upsampling (SMOTEENN, SMOTEENN Custom e SMOTE).

Tabella 9: Confronto complessivo delle metriche di performance dei modelli di classificazione sui tre dataset bilanciati.

Modello	SMOTEENN				SMOTEENN Custom				SMOTE			
	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1
Decision Tree Classifier	0.760	0.613	0.623	0.616	0.767	0.617	0.629	0.622	0.772	0.618	0.630	0.619
Random Forest Classifier	0.779	0.638	0.651	0.643	0.784	0.640	0.654	0.646	0.792	0.650	0.664	0.655
AdaBoost Classifier	0.773	0.537	0.616	0.558	0.761	0.527	0.596	0.546	0.777	0.542	0.622	0.562
Gradient Boosting Classifier	0.763	0.621	0.628	0.622	0.766	0.624	0.632	0.626	0.774	0.628	0.634	0.620
Neural Network (MLP)	0.761	0.612	0.625	0.616	0.769	0.618	0.632	0.624	0.770	0.609	0.624	0.611

Tabella 10: Metriche di valutazione dei modelli di classificazione sul dataset non bilanciato.

Modello	Accuracy	Precision (Macro Avg)	Recall (Macro Avg)	F1-Score (Macro Avg)
Random Forest	0.796	0.655	0.672	0.652
Gradient Boosting	0.794	0.627	0.655	0.631
Ada Boost	0.783	0.628	0.617	0.617
Decision Tree	0.761	0.589	0.598	0.593
Rete Neurale (MLP)	0.755	0.559	0.589	0.569

Dalla Tabella 9 emerge che il Random Forest Classifier risulta il modello con le migliori performance complessive su tutti e tre i dataset, con accuracy, precision, recall e F1-score macro più elevate. Le reti neurali (MLP) e i Decision Tree mostrano performance simili tra loro, leggermente inferiori rispetto alla Random Forest, ma comunque competitive, indicando che anche modelli più semplici o più complessi riescono a sfruttare efficacemente i dati bilanciati. L'AdaBoost Classifier tende ad avere precision più bassa pur mantenendo un recall relativamente alto, suggerendo una maggiore propensione a classificare correttamente alcune classi maggioritarie a scapito di errori su altre classi. Il Gradient Boosting Classifier si colloca in una posizione intermedia, con metriche bilanciate e coerenti. In generale le differenze tra le tecniche di bilanciamento influenzano leggermente le metriche macro.

Anche nella fase senza bilanciamento, il Random Forest Classifier ha raggiunto i migliori risultati, tra i più alti in assoluto, suggerendo che l'upsampling effettuato ha influito maggiormente su altri modelli come la Rete Neurale e il Decision Tree Classifier.

Dal punto di vista metodologico, i risultati ottenuti evidenziano come la qualità del bilanciamento dei dati sia determinante per le prestazioni di alcuni modelli di classificazione. Tecniche come **SMOTEENN** permettono di ridurre lo sbilanciamento tra le classi minoritarie, ma possono introdurre rumore e ambiguità nei confini decisionali, influenzando la capacità di generalizzazione. L'utilizzo di **SMOTEENN Custom**, senza downsampling della classe maggioritaria, preserva tutti

i campioni originali più rappresentativi, migliorando la stabilità delle metriche macro. Infine, **SMOTE** (upsampling al 50% della classe maggioritaria) mostra come un semplice oversampling delle classi minoritarie possa già migliorare precision, recall e F1-score macro, mantenendo la distribuzione originale dei dati e senza eliminare campioni potenzialmente utili.

Infine, si evidenzia che, nonostante le buone performance complessive, l'accuratezza da sola non è un indicatore sufficiente per valutare modelli su dataset sbilanciati: metriche come **F1-Score macro** e **AUC** risultano più affidabili per misurare la reale capacità del modello di distinguere tra le classi.

3 Clustering

L'obiettivo dell'analisi è quello di effettuare una Clusterizzazione dei distretti di polizia basati sui loro profili di criminalità, utilizzando algoritmi di clustering come **K-Means**.

Per l'analisi verranno utilizzate librerie **Python** come **pandas**, **matplotlib**, **seaborn**, **numpy**, **scipy** e **sklearn**, che consentiranno di effettuare una **pulizia accurata dei dati**, un' **analisi esplorativa** e la **modellazione del modello di clustering**.

3.1 Preprocessing e pulizia del dataset

Prima di applicare qualsiasi tecnica di clustering, è stata effettuata una fase di preprocessing dei dati con l'obiettivo di:

- rimuovere informazioni non rilevanti ai fini dell'analisi;
- uniformare il formato delle variabili;
- costruire feature coerenti e significative per gli algoritmi di clustering.

Il dataset originale contiene numerose colonne descrittive che non risultano utili per l'analisi dei profili di criminalità. Per questo motivo, sono state eliminate tutte le colonne non utilizzate direttamente nel processo di clustering, mantenendo solo le feature informative.

3.1.1 Rimozione delle colonne non necessarie

Sono state rimosse le seguenti colonne:

- `CrimeDate`, `CrimeCode`, `Location`;
- `District`, `Neighborhood`;
- `Total Incidents`, `Post`.

Queste variabili non contribuiscono alla definizione dei profili di evento utilizzati nel clustering oppure risultano ridondanti rispetto ad altre informazioni già presenti.

3.1.2 Gestione delle variabili temporali

La colonna `CrimeTime`, inizialmente espressa nel formato `HH:MM:SS`, è stata convertita estraendo esclusivamente l'ora dell'evento. Successivamente, l'ora è stata discretizzata in quattro fasce temporali:

- **Night:** 00:00 – 05:59
- **Morning:** 06:00 – 11:59
- **Afternoon:** 12:00 – 17:59
- **Evening:** 18:00 – 23:59

Questa trasformazione permette di ridurre la granularità temporale e di evidenziare pattern legati ai diversi momenti della giornata. Una volta creata la nuova variabile `HourBin`, la colonna `CrimeTime` è stata eliminata.

3.1.3 Estrazione delle coordinate geografiche

Le coordinate geografiche di latitudine e longitudine sono state estratte dalla colonna `Location_1`, che conteneva i valori in formato testuale. Dopo l'estrazione, la colonna originale è stata rimossa, mantenendo solo le due nuove feature numeriche:

- `Latitude`
- `Longitude`

3.1.4 Pulizia e normalizzazione delle variabili categoriche

La colonna `Description`, che descrive il tipo di reato, è stata uniformata rimuovendo eventuali spazi superflui. La variabile `Inside/Outside` è stata standardizzata per ridurre ambiguità, mappando tutti i valori in due sole categorie:

- `Inside`
- `Outside`

3.1.5 Gestione dei valori mancanti

Al termine delle trasformazioni, sono state eliminate tutte le righe contenenti valori nulli. Questa scelta garantisce che gli algoritmi di clustering operino su dati completi e coerenti.

3.1.6 Dataset finale

Dopo la fase di preprocessing, il dataset risultante contiene esclusivamente le feature rilevanti per l'analisi:

- coordinate geografiche (**Latitude**, **Longitude**);
- tipologia di reato (**Description**);
- arma utilizzata (**Weapon**);
- fascia oraria (**HourBin**);
- contesto dell'evento (**Inside/Outside**).

Questo dataset costituisce la base comune per tutte le successive analisi di clustering.

	Description	Inside/Outside	Weapon	Latitude	Longitude	HourBin
1	ROBBERY - COMMERCIAL	Inside	FIREARM	39.28242	-76.59288	Night
5	COMMON ASSAULT	Inside	HANDS	39.37070	-76.56709	Night
6	ROBBERY - COMMERCIAL	Outside	OTHER	39.28624	-76.64455	Night
9	COMMON ASSAULT	Inside	HANDS	39.30340	-76.63842	Morning
11	COMMON ASSAULT	Inside	HANDS	39.30587	-76.67216	Morning
12	COMMON ASSAULT	Inside	HANDS	39.29883	-76.61042	Morning
14	COMMON ASSAULT	Inside	HANDS	39.32120	-76.53906	Morning
17	AGG. ASSAULT	Outside	KNIFE	39.28810	-76.64888	Morning
19	ROBBERY - STREET	Outside	FIREARM	39.31249	-76.64539	Night
20	ROBBERY - STREET	Outside	FIREARM	39.31249	-76.64539	Night

Figura 14: Dataset processato

3.2 Preparazione dei dati per il clustering

Le variabili categoriche (**Description**, **Weapon**, **HourBin**, **Inside/Outside**) sono state trasformate tramite *label encoding*. Successivamente, tutte le feature (sia numeriche che categoriche codificate) sono state standardizzate mediante *StandardScaler*, in modo da garantire che ciascuna variabile contribuisca in maniera uniforme al calcolo delle distanze utilizzate dall'algoritmo KMeans.

Il clustering è stato effettuato utilizzando le seguenti 6 feature:

- **Latitude**
- **Longitude**

- Description_encoded
- Weapon_encoded
- HourBin_encoded
- Inside/Outside_encoded

È opportuno fare una precisazione metodologica riguardo l'uso del *Label Encoding* su variabili nominali. Sebbene questa codifica imponga un ordine numerico fittizio, l'utilizzo combinato di 6 dimensioni aiuta a compensare questo limite. La presenza di dati numerici reali (Latitudine e Longitudine) e la naturale tendenza dei reati a presentare pattern ricorrenti permettono all'algoritmo di raggruppare i dati in modo coerente, riducendo l'impatto dell'ordinamento arbitrario delle categorie.

	Latitude	Longitude	Description_encoded	Weapon_encoded	HourBin_encoded	Inside/Outside_encoded
0	-0.434227	0.587980	1.357305	-1.327115	1.501239	-0.913462
1	1.111919	1.208409	-0.246169	-0.245826	1.501239	-0.913462
2	-0.367323	-0.655043	1.357305	1.916752	1.501239	1.094736
3	-0.066781	-0.507574	-0.246169	-0.245826	0.601171	-0.913462
4	-0.023521	-1.319256	-0.246169	-0.245826	0.601171	-0.913462

Figura 15: Encoded Dataset

3.3 Clustering degli eventi senza filtraggio DBSCAN

In questa sezione viene descritto il processo di clustering applicato direttamente al dataset preprocessato, senza l'utilizzo di tecniche di filtraggio preliminare. L'obiettivo è identificare profili di criminalità ricorrenti sulla base delle caratteristiche degli stessi.

3.3.1 Scelta del numero di cluster

Per determinare il numero ottimale di cluster, sono stati utilizzati due criteri complementari:

- il metodo dell'**Elbow**;
- l'**indice di Silhouette**.

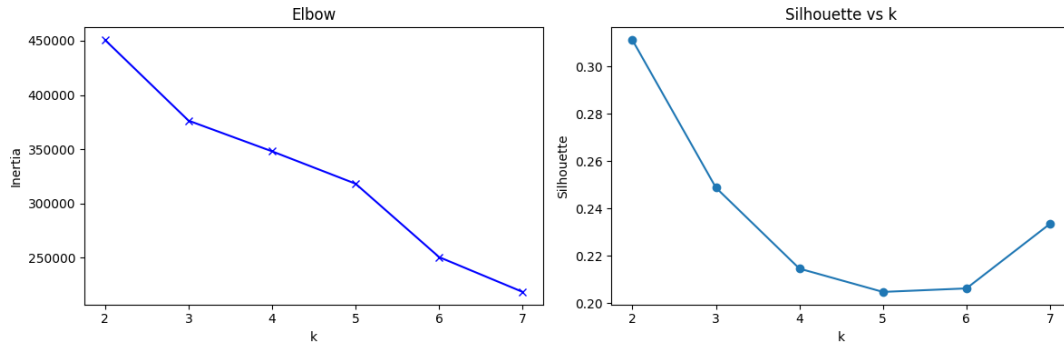


Figura 16: Metodo dell'Elbow e indice di Silhouette per la scelta del numero di cluster.

Il grafico dell'Elbow mostra una riduzione significativa dell'inertia all'aumentare del numero di cluster fino a $k = 6$. Oltre questo valore, il miglioramento diventa progressivamente meno marcato, suggerendo che l'aumento della complessità del modello non porta benefici sostanziali.

L'analisi dell'indice di Silhouette ci mostra invece che: il valore massimo si registra per $k = 2$, ma tale soluzione risulta troppo grossolana per descrivere la varietà dei comportamenti criminali presenti nel dataset. I valori di Silhouette da $k = 4$ a $k = 6$ tendono a decrescere ulteriormente indicando una performance inferiore, anche se di poco, del modello all'aumentare del numero di cluster.

Sulla base di queste considerazioni, è stato scelto un numero di cluster pari a:

$$k = 4$$

poiché rimane comunque adeguato e rappresenta un buon compromesso tra qualità del clustering e interpretabilità.

3.3.2 Applicazione di KMeans

Una volta fissato il valore di k , l'algoritmo KMeans è stato applicato all'intero dataset. Il risultato è una suddivisione degli eventi in quattro cluster, ciascuno contenente un numero significativo di osservazioni, a conferma della capacità del modello di intercettare pattern ricorrenti all'interno dei dati.

- **Cluster 0:** 36.267
- **Cluster 1:** 13.397
- **Cluster 2:** 24.364
- **Cluster 3:** 16.847

3.3.3 Visualizzazione dei cluster tramite PCA

Per facilitare l'interpretazione dei risultati, è stata applicata una *Principal Component Analysis* (PCA) con due componenti principali.

La PCA consente di:

- ridurre la dimensionalità dei dati,
- proiettare gli eventi in uno spazio a due dimensioni,
- analizzare quali variabili contribuiscono maggiormente alla separazione dei cluster.

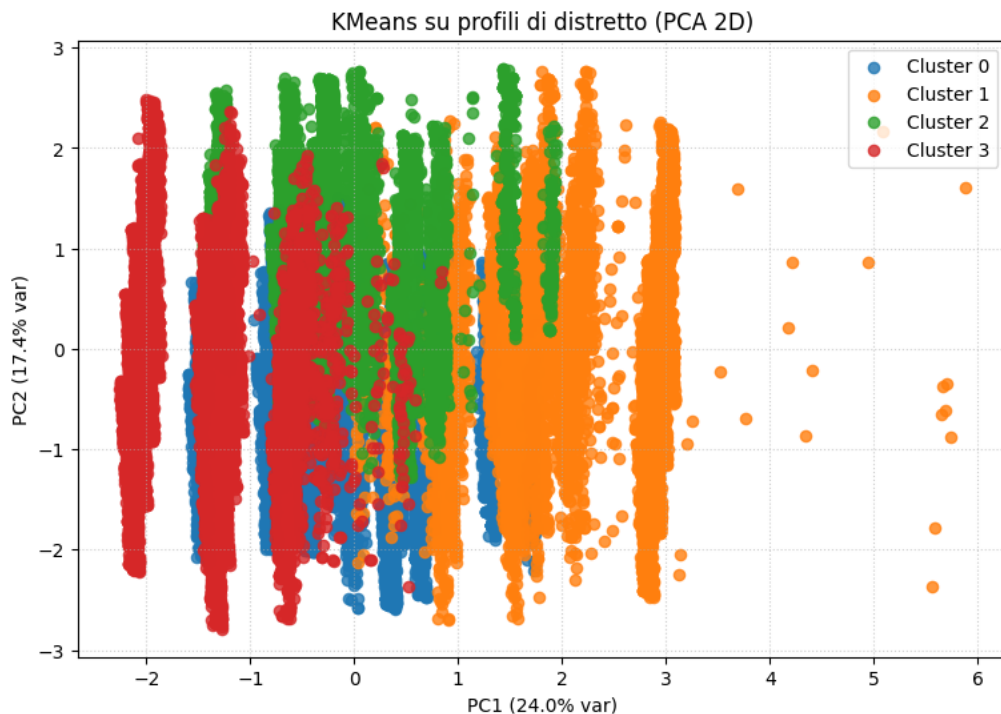


Figura 17: Visualizzazione dei cluster mediante PCA a due componenti.

Le prime due componenti spiegano complessivamente circa il 41% della varianza totale ($\sim 24\%$ per la prima componente e $\sim 17\%$ per la seconda).

L'analisi dei *loadings* (lst:2) mostra che:

- la prima componente principale (PC1) è fortemente influenzata dal tipo di reato, dal contesto *inside/outside* e dal tipo di arma utilizzata;
- la seconda componente (PC2) è principalmente legata alla fascia oraria e, secondariamente, alla posizione geografica.

```

1  === PC1 (24.0%) - Cosa sposta i punti a destra/sinistra? ===
2  --- Spingono a DESTRA (Valori Alti) ---
3  Description_encoded 0.681552
4  Inside/Outside_encoded 0.395460
5  HourBin_encoded 0.076061
6  Latitude 0.071252
7  Longitude -0.004945
8
9  --- Spingono a SINISTRA (Valori Bassi) ---
10 Weapon_encoded -0.606804
11 Longitude -0.004945
12 Latitude 0.071252
13 HourBin_encoded 0.076061
14 Inside/Outside_encoded 0.395460
15
16 === PC2 (17.4%) - Cosa sposta i punti in alto/basso? ===
17 --- Spingono in ALTO (Valori Alti) ---
18 HourBin_encoded 0.710886
19 Longitude 0.638431
20 Description_encoded 0.001901
21 Latitude -0.013333
22 Weapon_encoded -0.096922
23
24 --- Spingono in BASSO (Valori Bassi) ---
25 Inside/Outside_encoded -0.278339
26 Weapon_encoded -0.096922
27 Latitude -0.013333
28 Description_encoded 0.001901
29 Longitude 0.638431

```

Listing 2: Stampa delle feature più influenti per PC1 (Asse X) e PC2 (Asse Y)

La rappresentazione bidimensionale evidenzia una parziale sovrapposizione tra i cluster, coerente con la natura eterogenea dei dati, ma consente comunque di osservare una struttura non casuale nella distribuzione degli eventi(dovuto probabilmente all'encoding seguito dallo scaling).

3.3.4 Profilazione dei cluster

Per rendere i cluster interpretabili, è stata effettuata una fase di profilazione. Dopo aver riportato le variabili categoriche tramite *one-hot encoding*, sono stati calcolati i valori medi per ciascun cluster. In questo modo, ogni cluster può essere interpretato come un profilo percentuale di tipologie di reato, armi utilizzate, fasce orarie e contesto dell'evento.

Tabella 11: Profili percentuali dei cluster

Feature	Cluster 0	Cluster 1	Cluster 2	Cluster 3
Description_AGG. ASSAULT	0.185	0.000	0.230	0.884
Description_COMMON ASSAULT	0.788	0.001	0.740	0.000
Description_RAPE	0.000	0.000	0.000	0.087
Description_ROBBERY - CARJACKING	0.004	0.044	0.006	0.006
Description_ROBBERY - COMMERCIAL	0.023	0.104	0.020	0.015
Description_ROBBERY - RESIDENCE	0.000	0.088	0.003	0.009
Description_ROBBERY - STREET	0.000	0.763	0.000	0.000
Weapon_FIREARM	0.118	0.773	0.093	0.000
Weapon_HANDS	0.848	0.001	0.804	0.000
Weapon_KNIFE	0.034	0.142	0.102	0.239
Weapon_OTHER	0.000	0.084	0.001	0.761
HourBin_Night	0.000	0.247	0.621	0.151
HourBin_Morning	0.000	0.115	0.379	0.147
HourBin_Afternoon	0.493	0.175	0.000	0.285
HourBin_Evening	0.507	0.464	0.000	0.417
Inside/Outside_Inside	0.598	0.232	0.663	0.510
Inside/Outside_Outside	0.402	0.768	0.337	0.490

3.3.5 Analisi visiva dei profili di cluster tramite heatmap

La heatmap consente un confronto immediato tra i cluster, evidenziando le caratteristiche dominanti di ciascun profilo.

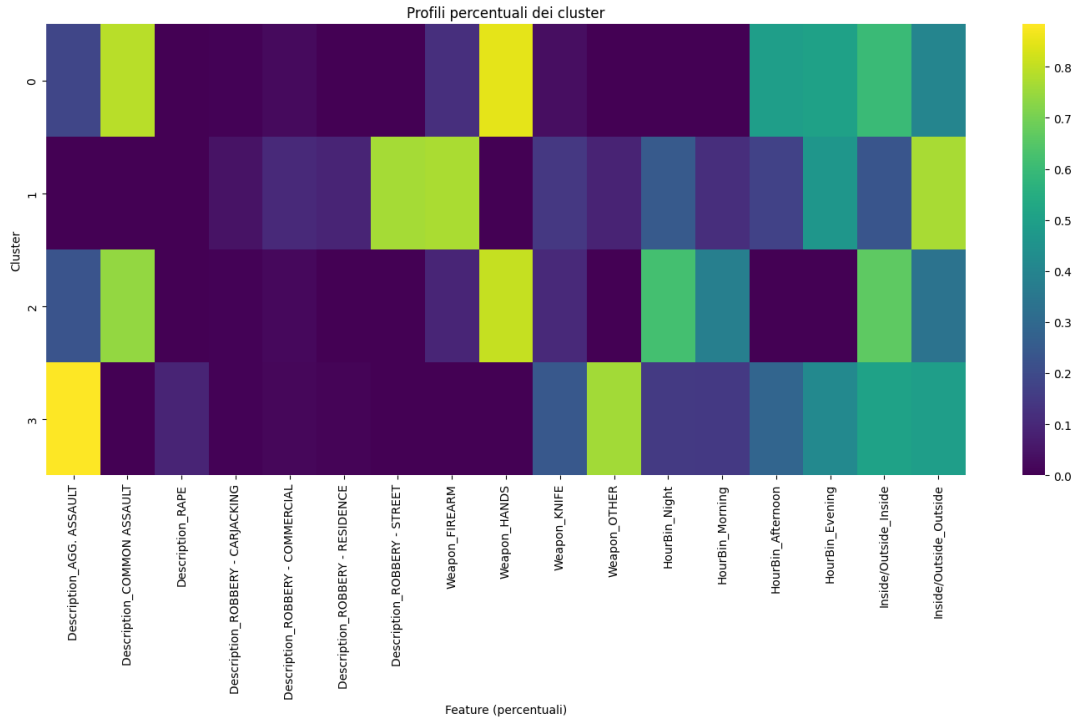


Figura 18: Heatmap dei profili percentuali dei cluster.

La heatmap costruita rappresenta i **profili percentuali dei cluster**, ottenuti aggregando i dati e calcolando la media delle variabili categoriali per ciascun gruppo. Ogni cella indica quindi la **frequenza relativa** di una determinata categoria all'interno del cluster corrispondente.

Questa visualizzazione consente un confronto immediato tra i profili dei gruppi individuati, mettendo in evidenza similitudini e differenze rispetto a: *tipologia di reato, tipo di arma, fascia oraria e contesto (interno o esterno)*.

Cluster 0 Il **Cluster 0** è caratterizzato da:

- una netta prevalenza di *Common Assault*;
- utilizzo quasi esclusivo di *mani come armi (Hands)*;
- concentrazione degli eventi nelle fasce *pomeridiane e serali*;
- maggiore incidenza di eventi avvenuti *all'interno (Inside)*.

Questo profilo suggerisce reati di aggressione non armata, diffusi durante il giorno, in contesti prevalentemente chiusi.

Cluster 1 Il **Cluster 1** mostra un profilo marcatamente distinto:

- alta incidenza di *Robbery – Street*;
- uso prevalente di *armi da fuoco (Firearm)*;
- eventi concentrati soprattutto in orari *serali e notturni*;
- netta prevalenza di eventi *all'esterno (Outside)*.

Questo cluster identifica una tipologia ben definita di **rapine armate**, con modalità operative più violente rispetto agli altri gruppi.

Cluster 2 Il **Cluster 2** condivide alcune somiglianze con il cluster 0, ma presenta caratteristiche distintive:

- prevalenza combinata di *Common Assault* e *Agg. Assault*;
- uso predominante di *Hands* come arma;
- maggiore concentrazione di eventi nelle *ore notturne*;
- elevata incidenza di eventi *all'interno*.

Questo gruppo rappresenta aggressioni non armate simili a quelle del cluster 0, ma con una **diversa distribuzione temporale**, spostata maggiormente verso la notte.

Cluster 3 Il **Cluster 3** si differenzia dagli altri per:

- forte presenza di *Agg. Assault*;
- utilizzo più frequente di armi classificate come *Other* o *Knife*;
- distribuzione temporale relativamente uniforme, con una leggera prevalenza in *fascia serale*;
- bilanciamento tra eventi *Inside* e *Outside*.

Il profilo suggerisce una categoria di eventi più **eterogenea**, potenzialmente legata a episodi di maggiore gravità o complessità.

3.3.6 Analisi tramite lift

Poiché i valori medi tendono a privilegiare le categorie più frequenti nel dataset, è stata introdotta un'analisi basata sul **lift**. Il lift misura quanto una determinata caratteristica è più (o meno) rappresentata in un cluster rispetto alla media globale del dataset.

Questa analisi permette di identificare gli elementi realmente distintivi di ciascun cluster, anche quando tali elementi sono poco frequenti in termini assoluti.

```
1  === 0: feature con lift pi alto ===
2  HourBin_Afternoon 2.07
3  Description_COMMON ASSAULT 2.06
4  Weapon_HANDS 2.05
5  HourBin_Evening 1.46
6  Inside/Outside_Inside 1.19
7  Inside/Outside_Outside 0.81
8  Description_AGG. ASSAULT 0.57
9  Description_ROBBERY - COMMERCIAL 0.56
10 Weapon_FIREARM 0.48
11 Description_ROBBERY - CARJACKING 0.29
12 Weapon_KNIFE 0.26
13 Description_ROBBERY - RESIDENCE 0.00
14 Description_ROBBERY - STREET 0.00
15 Weapon_OTHER 0.00
16 HourBin_Night 0.00
17
18  === 1: feature con lift pi alto ===
19 Description_ROBBERY - STREET 4.00
20 Description_ROBBERY - RESIDENCE 3.52
21 Weapon_FIREARM 3.14
22 Description_ROBBERY - CARJACKING 2.93
23 Description_ROBBERY - COMMERCIAL 2.56
24 Inside/Outside_Outside 1.54
25 HourBin_Evening 1.34
26 Weapon_KNIFE 1.10
27 HourBin_Night 0.97
28 HourBin_Afternoon 0.73
29 HourBin_Morning 0.72
30 Inside/Outside_Inside 0.46
31 Weapon_OTHER 0.40
32 Description_COMMON ASSAULT 0.00
```

```

33 Weapon_HANDS 0.00
34
35 === 2: feature con lift pi alto ===
36 HourBin_Night 2.44
37 HourBin_Morning 2.37
38 Weapon_HANDS 1.95
39 Description_COMMON ASSAULT 1.94
40 Inside/Outside_Inside 1.32
41 Weapon_KNIFE 0.79
42 Description_AGG. ASSAULT 0.71
43 Inside/Outside_Outside 0.67
44 Description_ROBBERY - COMMERCIAL 0.50
45 Description_ROBBERY - CARJACKING 0.40
46 Weapon_FIREARM 0.38
47 Description_ROBBERY - RESIDENCE 0.13
48 Weapon_OTHER 0.00
49 Description_ROBBERY - STREET 0.00
50 HourBin_Afternoon 0.00
51
52 === 3: feature con lift pi alto ===
53 Description Rape 4.00
54 Weapon_OTHER 3.60
55 Description_AGG. ASSAULT 2.72
56 Weapon_KNIFE 1.85
57 HourBin_Evening 1.20
58 HourBin_Afternoon 1.20
59 Inside/Outside_Inside 1.02
60 Inside/Outside_Outside 0.98
61 HourBin_Morning 0.92
62 HourBin_Night 0.59
63 Description_ROBBERY - COMMERCIAL 0.38
64 Description_ROBBERY - CARJACKING 0.37
65 Description_ROBBERY - RESIDENCE 0.35
66 Description_ROBBERY - STREET 0.00
67 Description_COMMON ASSAULT 0.00

```

Dall'analisi del lift emergono profili differenziati, come suggerito dalla heatmap:

- **Cluster 0:** caratterizzato prevalentemente da aggressioni comuni con uso delle mani e concentrate nelle ore pomeridiane e serali;

- **Cluster 1:** fortemente associato a rapine (street, residence, carjacking), con uso di armi da fuoco e contesto prevalentemente esterno;
- **Cluster 2:** dominato da eventi notturni e mattutini, con aggressioni comuni e contesto interno;
- **Cluster 3:** distinto per reati di tipo RAPE e uso di armi non convenzionali, con una distribuzione temporale più equilibrata.

L'utilizzo del lift consente quindi una lettura più approfondita e mirata dei risultati del clustering rispetto alla sola analisi delle frequenze, poichè mette in evidenza anche le feature meno rappresentate del dataset e le utilizza per caratterizzare i singoli cluster.

3.4 Clustering degli eventi con filtraggio DBSCAN

In questa sezione viene descritto il processo di clustering applicato al dataset dopo una fase preliminare di filtraggio tramite l'algoritmo DBSCAN. L'obiettivo di questo approccio è ridurre l'influenza del rumore e degli eventi isolati, concentrando l'analisi sulle aree a maggiore densità di eventi criminali.

3.4.1 Filtraggio preliminare con DBSCAN

Prima dell'applicazione di KMeans, è stato introdotto un filtraggio dei dati tramite l'algoritmo **DBSCAN** (*Density-Based Spatial Clustering of Applications with Noise*). L'obiettivo di questo passaggio è rimuovere gli *outlier* e conservare solo gli eventi localizzati in regioni ad alta densità, ossia quelli che rappresentano pattern più consistenti e strutturati nel dataset.

DBSCAN è stato scelto perché:

- consente di individuare cluster di forma arbitraria, basati sulla densità locale dei punti;
- permette di etichettare come *rumore* (cluster -1) gli eventi isolati o non riconducibili a gruppi densi;
- restituisce solo i punti che appartengono a cluster compatti, più informativi per l'analisi successiva.

Scelta del parametro ϵ Per stimare il parametro ϵ , è stato utilizzato il **K-Distance Plot**, che rappresenta per ciascun punto del dataset la distanza dal suo k -esimo vicino più prossimo (nel nostro caso, il 40-esimo). I punti sono ordinati in modo crescente rispetto a tale distanza.

Nel grafico ottenuto (Figura 19), è possibile osservare tre zone distinte:

- **Regione iniziale a bassa pendenza**, dove le distanze crescono lentamente: rappresenta punti in regioni ad alta densità (candidati *core points*);
- **Punto di flesso (elbow)**, osservabile tra 0.22 e 0.25, dove la curva cambia inclinazione in modo netto: identifica la soglia di separazione tra zone dense e punti isolati;
- **Coda finale a pendenza elevata**, in cui le distanze crescono rapidamente: rappresenta eventi isolati o rumorosi.

Sulla base di questa analisi visiva, è stato scelto un valore di ϵ pari a **0.25**, in quanto rappresenta un buon compromesso tra l'inclusione di regioni dense e l'esclusione del rumore.

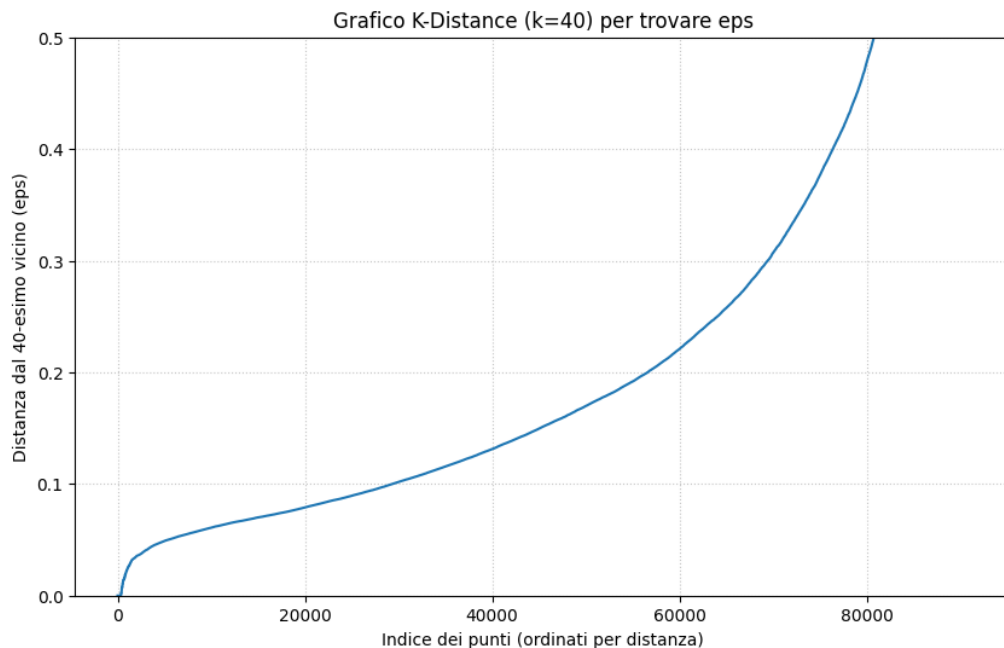


Figura 19: K-Distance Plot

Filtraggio del rumore Impostati i parametri $\epsilon = 0.25$ e $\text{min_samples} = 40$, DBSCAN ha etichettato come rumore circa il **21%** degli eventi del dataset. Tutti i punti con etichetta -1 sono stati rimossi, ottenendo un nuovo dataset costituito esclusivamente da eventi appartenenti a cluster densi.

```

1 Punti per cluster: (-1 = Rumore/outlier)
2 cluster
3 -1 19396
4 8 8920
5 7 8475
6 0 6738
7 11 5867
8 ...
9 79 40
10 83 40
11 77 40
12 73 26
13 74 26
14
15 Cluster trovati (escluso rumore): 85
16 Percentuale di rumore: 21.3%

```

Il dataset risultante, dopo aver escluso gli eventi etichettati come rumore, è stato quindi utilizzato come base per l'algoritmo KMeans. Questo passaggio consente di lavorare su un insieme di dati più compatto e informativo, riducendo l'influenza di osservazioni isolate che potrebbero compromettere la qualità del clustering.

3.4.2 Scelta del numero di cluster

Sul dataset filtrato è stato applicato nuovamente il metodo dell'Elbow e l'indice di Silhouette per determinare il numero ottimale di cluster.

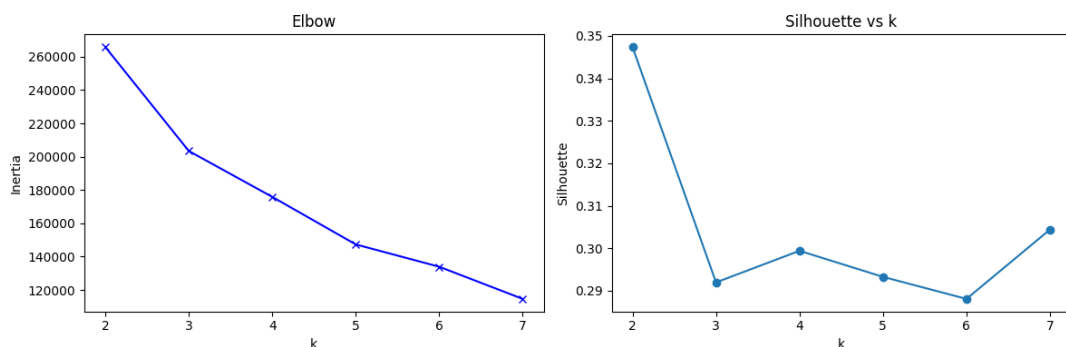


Figura 20: Metodo dell'Elbow e indice di Silhouette sul dataset filtrato tramite DBSCAN.

Il numero ottimale di cluster k è stato scelto analizzando congiuntamente i grafici di *Elbow* e *Silhouette*, ottenuti sul dataset filtrato da DBSCAN.

Analisi dell'Elbow Method. Dal grafico dell'inertia si osserva:

- una forte riduzione dell'inertia passando da $k = 2$ a $k = 3$;
- una diminuzione ancora significativa fino a $k = 5$;
- oltre $k = 5$, la curva tende a diventare più lineare, indicando benefici marginali sempre più ridotti.

Questo andamento suggerisce la presenza di un punto di flesso attorno a $k = 5$, oltre il quale l'aumento del numero di cluster non porta a una separazione sostanzialmente migliore dei dati.

Analisi del Silhouette Score. Il Silhouette Score mostra:

- il valore massimo per $k = 2$, comportamento atteso poiché una suddivisione binaria tende a produrre cluster più separati in modo artificiale;
- un valore relativamente elevato e stabile per $k = 4$;
- oscillazioni e lievi peggioramenti per valori di $k > 4$.

In particolare, $k = 4$ rappresenta un buon compromesso tra qualità della separazione dei cluster, stabilità del risultato e interpretabilità dei gruppi ottenuti.

Scelta finale di k . Rispetto al caso non filtrato, il filtraggio tramite DBSCAN riduce l'influenza del rumore e degli outlier, rendendo la struttura dei dati più compatta e producendo una curva dell'Elbow più regolare e interpretabile. Alla luce di queste considerazioni, è stato scelto:

$$k = 4$$

Una scelta che risulta coerente sul dataset filtrato e, allo stesso tempo, utile per mantenere un confronto diretto con l'analisi precedente.

3.4.3 Applicazione di KMeans sul dataset filtrato

L'algoritmo KMeans è stato applicato al dataset filtrato, ottenendo quattro cluster composti esclusivamente da eventi appartenenti a regioni ad alta densità.

Rispetto al clustering senza filtraggio, l'assegnazione degli eventi ai cluster cambia sensibilmente: dopo DBSCAN alcuni cluster risultano molto più piccoli. Questo è coerente con l'idea che il filtraggio abbia rimosso eventi isolati e casi meno coerenti, lasciando in input a K-Means una struttura più 'pulita' e maggiormente concentrata sui pattern più densi.

Distribuzione degli eventi nei cluster.

- **Cluster 0:** 34.578 eventi
- **Cluster 1:** 6.752 eventi
- **Cluster 2:** 8.738 eventi
- **Cluster 3:** 21.411 eventi

La distribuzione non è uniforme: alcuni cluster (in particolare il Cluster 0) raccolgono una quota maggiore di eventi, suggerendo la presenza di un pattern più frequente o più “generico” nel dataset filtrato. Al contrario, i cluster più piccoli (Cluster 1 e Cluster 2) potrebbero rappresentare modalità di crimine più specifiche o meno ricorrenti.

3.4.4 Visualizzazione e interpretazione dei cluster tramite PCA

Per supportare l’interpretazione dei cluster ottenuti con l’algoritmo K-Means, viene applicata la **Principal Component Analysis (PCA)** al dataset standardizzato e filtrato.

È importante sottolineare che la PCA **non viene utilizzata per il clustering**, ma esclusivamente come strumento di analisi e interpretazione dei risultati prodotti da K-Means.

In questa analisi viene mostrata la proiezione sulle **prime due componenti principali**, che spiegano la quota maggiore di varianza complessiva del dataset.

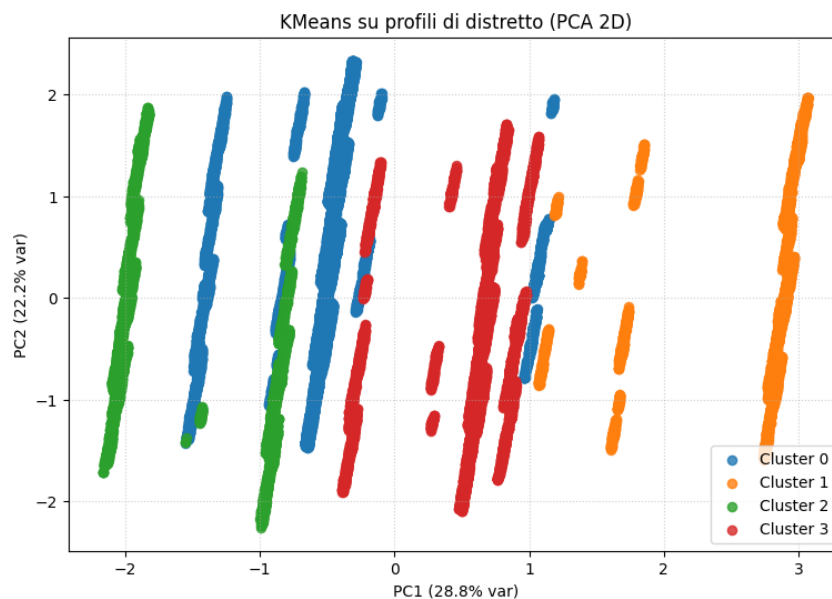


Figura 21: Proiezione PCA (2D) degli eventi filtrati, colorati in base al cluster K-Means

Analisi della proiezione PCA in due dimensioni Il grafico mostra la distribuzione degli eventi lungo le prime due componenti principali (PC1 e PC2), con i punti colorati in base al cluster di appartenenza.

Visivamente, i cluster risultano solo parzialmente separati e organizzati lungo bande quasi verticali. Questo comportamento è coerente con la natura dei dati:

- molte feature sono categoriche e sono state trasformate tramite *label encoding*,
- la PCA applica combinazioni lineari di tali variabili,
- la separazione tra cluster non è necessariamente netta nello spazio bidimensionale.

Di conseguenza, il valore informativo principale della PCA non risiede tanto nella separazione visiva dei punti, quanto nell'**analisi dei contributi delle variabili alle componenti principali**, illustrata di seguito.

Interpretazione delle componenti principali Le componenti principali permettono di comprendere **quali variabili guidano la struttura dei cluster**.

```
1 === PC1 (28.8%) - Cosa sposta i punti a destra/sinistra? ===
2 --- Spingono a DESTRA (Valori Alti) ---
3 Description_encoded 0.610927
4 Inside/Outside_encoded 0.568975
5 HourBin_encoded 0.062057
6 Longitude 0.043803
7 Latitude 0.016133
8
9 --- Spingono a SINISTRA (Valori Bassi) ---
10 Weapon_encoded -0.544982
11 Latitude 0.016133
12 Longitude 0.043803
13 HourBin_encoded 0.062057
14 Inside/Outside_encoded 0.568975
15
16 === PC2 (22.2%) - Cosa sposta i punti in alto/basso? ===
17 --- Spingono in ALTO (Valori Alti) ---
18 HourBin_encoded 0.880935
19 Longitude 0.323281
20 Description_encoded 0.057944
21 Latitude -0.010016
22 Weapon_encoded -0.135328
```

```

23
24 --- Spingono in BASSO (Valori Bassi) ---
25 Inside/Outside_encoded -0.312524
26 Weapon_encoded -0.135328
27 Latitude -0.010016
28 Description_encoded 0.057944
29 Longitude 0.323281

```

PC1 – Asse orizzontale (28.8% della varianza) La prima componente principale è fortemente influenzata da:

- `Description_encoded` (contributo positivo elevato),
- `Inside/Outside_encoded`,
- in misura minore da `HourBin_encoded`.

Valori elevati di PC1 sono quindi associati a specifiche tipologie di reato e a una marcata distinzione tra eventi avvenuti **all'interno o all'esterno**.

Al contrario, valori bassi di PC1 sono principalmente guidati da:

- `Weapon_encoded` (contributo negativo rilevante),

suggerendo che questa componente separa gli eventi in base alle **modalità del crimine e al contesto**, più che alla posizione geografica.

PC2 – Asse verticale (22.2% della varianza) La seconda componente principale è dominata da:

- `HourBin_encoded`, che presenta il contributo più elevato in assoluto.

Questo indica che PC2 rappresenta principalmente una **dimensione temporale**, distinguendo gli eventi in base alla fascia oraria in cui si verificano.

Contributi secondari sono dati da:

- `Longitude` e, in misura minore, `Description_encoded`,

mentre `Inside/Outside_encoded` contribuisce negativamente, spostando i punti verso valori più bassi dell'asse.

Nel complesso:

- **PC1** cattura differenze legate al **tipo di reato e al contesto (interno/esterno)**;

- **PC2** cattura differenze legate prevalentemente al **momento temporale del crimine**.

Questo conferma che i cluster K-Means sono guidati principalmente da **caratteristiche comportamentali e temporali**, più che dalla sola posizione geografica.

3.4.5 Profilazione dei cluster

Anche in questo caso, per rendere interpretabili i cluster, è stata effettuata una profilazione basata su:

- tipologia di reato;
- arma utilizzata;
- fascia oraria;
- contesto *inside/outside*.

Dopo l'applicazione del *one-hot encoding*, sono stati calcolati i valori medi per ciascun cluster, ottenendo una rappresentazione percentuale delle caratteristiche dominanti.

Tabella 12: Profili percentuali dei cluster (K-Means dopo filtraggio DBSCAN)

Feature	Cluster 0	Cluster 1	Cluster 2	Cluster 3
Description_AGG. ASSAULT	11.4%	0.0%	99.3%	24.3%
Description_COMMON ASSAULT	86.3%	0.0%	0.0%	75.7%
Description_RAPE	0.0%	0.0%	0.7%	0.0%
Description_ROBBERY - COMMERCIAL	2.2%	0.0%	0.0%	0.0%
Description_ROBBERY - RESIDENCE	0.0%	0.8%	0.0%	0.0%
Description_ROBBERY - STREET	0.0%	99.2%	0.0%	0.0%
Weapon_FIREARM	3.3%	86.9%	0.0%	12.4%
Weapon_HANDS	88.8%	0.0%	0.0%	77.3%
Weapon_KNIFE	7.9%	9.6%	0.0%	10.3%
Weapon_OTHER	0.0%	3.6%	100.0%	0.0%
HourBin_Night	22.7%	29.3%	18.8%	22.4%
HourBin_Morning	17.6%	3.6%	10.1%	8.3%
HourBin_Afternoon	27.6%	14.9%	30.5%	30.7%
HourBin_Evening	32.1%	52.3%	40.5%	38.6%
Inside/Outside_Inside	100.0%	0.8%	50.8%	0.0%
Inside/Outside_Outside	0.0%	99.2%	49.2%	100.0%

3.4.6 Analisi dei profili di cluster tramite heatmap

Per confrontare in modo immediato i profili dei cluster ottenuti con K-Means, viene utilizzata una *heatmap* basata sui valori percentuali delle feature aggregate per cluster.

La heatmap rappresenta:

- sulle righe i cluster individuati dall'algoritmo;
- sulle colonne le feature categoriali trasformate tramite one-hot encoding;
- tramite una scala cromatica l'intensità (percentuale) di ciascuna feature all'interno del cluster.

Questa visualizzazione consente un confronto rapido tra i cluster e permette di individuare pattern ricorrenti nelle tipologie di reato, nelle armi utilizzate, nel contesto *Inside/Outside* e nelle fasce orarie.

In Figura 22 è riportata la heatmap dei profili percentuali dei cluster.

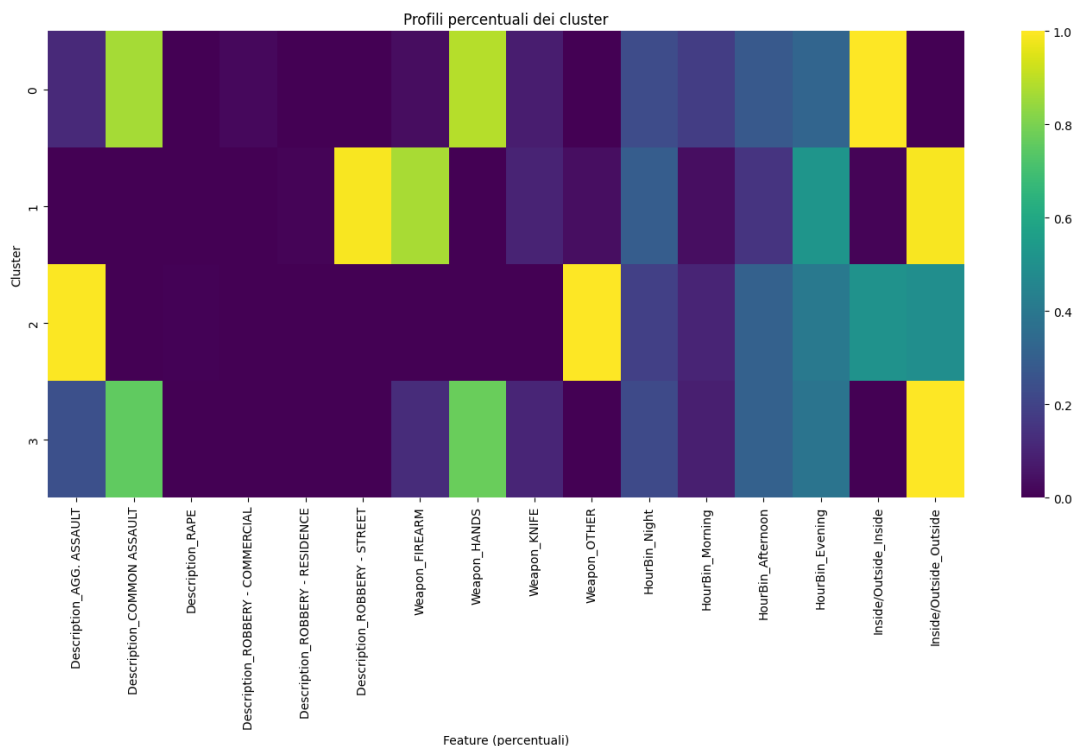


Figura 22: Heatmap dei profili percentuali dei cluster con filtraggio DBSCAN.

Dalla heatmap emergono pattern chiari e coerenti con quanto osservato nei passaggi precedenti:

- alcuni cluster risultano dominati da specifiche tipologie di reato (ad esempio aggressioni comuni o rapine);
- la distribuzione delle armi e del contesto *Inside/Outside* evidenzia differenze marcate tra i cluster;
- le fasce orarie mostrano una certa variabilità, ma con trend complessivamente regolari.

Tuttavia, questa rappresentazione mette in evidenza **soprattutto le categorie più frequenti in assoluto** nel dataset. Di conseguenza:

- variabili rare (come RAPE, ROBBERY - COMMERCIAL o alcune tipologie di arma) appaiono poco rilevanti nella heatmap;
- non è possibile distinguere se una feature sia realmente *caratteristica* di un cluster o semplicemente comune a tutti.

Per superare questo limite e ottenere una caratterizzazione più informativa dei cluster, viene introdotta l'analisi del **Lift**, che confronta la probabilità di ciascuna feature all'interno del cluster con la media globale del dataset.

3.4.7 Analisi tramite lift

Per identificare le caratteristiche realmente distintive di ciascun cluster, è stata applicata l'analisi del **lift**, confrontando la distribuzione delle feature all'interno di ogni cluster con la media globale del dataset filtrato.

```

1 === 0: feature con lift piu' alto ===
2 Description_ROBBERY - COMMERCIAL 4.00
3 Inside/Outside_Inside 2.64
4 Weapon_HANDS 2.14
5 Description_COMMON ASSAULT 2.13
6 HourBin_Morning 1.78
7 Weapon_KNIFE 1.14
8 HourBin_Afternoon 1.06
9 HourBin_Night 0.97
10 HourBin_Evening 0.78
11 Description_AGG. ASSAULT 0.34
12 Weapon_FIREARM 0.13
13 Description_RAPE 0.00
14 Description_ROBBERY - RESIDENCE 0.00

```

```

15 Description_ROBBERY - STREET 0.00
16 Weapon_OTHER 0.00
17
18 === 1: feature con lift piu' alto ===
19 Description_ROBBERY - RESIDENCE 4.00
20 Description_ROBBERY - STREET 4.00
21 Weapon_FIREARM 3.39
22 Inside/Outside_Outside 1.60
23 Weapon_KNIFE 1.38
24 HourBin_Evening 1.28
25 HourBin_Night 1.26
26 HourBin_Afternoon 0.57
27 HourBin_Morning 0.36
28 Weapon_OTHER 0.14
29 Inside/Outside_Inside 0.02
30 Description_AGG. ASSAULT 0.00
31 Description_COMMON ASSAULT 0.00
32 Description_RAPE 0.00
33 Description_ROBBERY - COMMERCIAL 0.00
34
35 === 2: feature con lift piu' alto ===
36 Description_RAPE 4.00
37 Weapon_OTHER 3.86
38 Description_AGG. ASSAULT 2.94
39 Inside/Outside_Inside 1.34
40 HourBin_Afternoon 1.18
41 HourBin_Morning 1.02
42 HourBin_Evening 0.99
43 HourBin_Night 0.81
44 Inside/Outside_Outside 0.79
45 Weapon_KNIFE 0.00
46 Description_COMMON ASSAULT 0.00
47 Description_ROBBERY - COMMERCIAL 0.00
48 Description_ROBBERY - RESIDENCE 0.00
49 Description_ROBBERY - STREET 0.00
50 Weapon_FIREARM 0.00
51
52 === 3: feature con lift piu' alto ===
53 Description_COMMON ASSAULT 1.87
54 Weapon_HANDS 1.86

```

55	Inside/Outside_Outside	1.61
56	Weapon_KNIFE	1.48
57	HourBin_Afternoon	1.18
58	HourBin_Night	0.96
59	HourBin_Evening	0.94
60	HourBin_Morning	0.84
61	Description_AGG. ASSAULT	0.72
62	Weapon_FIREARM	0.48
63	Description_RAPE	0.00
64	Description_ROBBERY - COMMERCIAL	0.00
65	Description_ROBBERY - RESIDENCE	0.00
66	Description_ROBBERY - STREET	0.00
67	Weapon_OTHER	0.00

L'analisi del lift mette in evidenza profili molto marcati:

- **Cluster 0:** fortemente associato a rapine commerciali, con contesto prevalentemente interno e uso delle mani;
- **Cluster 1:** dominato da rapine in strada e in residenza, con un forte utilizzo di armi da fuoco e prevalenza di eventi serali e notturni;
- **Cluster 2:** caratterizzato da reati di tipo **RAPE**, con uso di armi non convenzionali e una distribuzione temporale più equilibrata;
- **Cluster 3:** associato ad aggressioni comuni, con uso delle mani e contesto prevalentemente esterno.

Rispetto all'analisi senza filtraggio, il lift mostra valori simili ma più estremi, segnale che il filtraggio DBSCAN ha contribuito a rendere i cluster più omogenei e maggiormente rappresentativi di specifici profili criminali.

3.5 Confronto tra clustering con e senza filtraggio DBSCAN

In questa sezione vengono messi a confronto i risultati ottenuti applicando K-Means:

- direttamente sull'intero dataset;
- sul dataset precedentemente filtrato tramite DBSCAN.

L'obiettivo del confronto non è valutare quale approccio sia “migliore” in senso assoluto, ma comprendere **come il filtraggio del rumore influenzi la struttura dei cluster e la loro interpretabilità**.

3.5.1 Struttura e distribuzione dei cluster

Nel clustering **senza filtraggio**, tutti gli eventi, inclusi quelli rari, isolati o atipici, contribuiscono alla definizione dei centroidi di K-Means. Questo porta a cluster:

- numericamente più bilanciati;
- ma spesso più eterogenei al loro interno;
- con centroidi influenzati anche da eventi poco rappresentativi.

Nel clustering **con filtraggio DBSCAN**, invece:

- gli eventi isolati e il rumore vengono rimossi;
- K-Means opera solo su regioni ad alta densità;
- i cluster risultano meno influenzati da outlier.

La distribuzione degli eventi nei cluster filtrati risulta più sbilanciata, ma questo comportamento è coerente con la presenza di:

- cluster “generici”, che raccolgono pattern molto frequenti;
- cluster più piccoli, che rappresentano modalità di crimine specifiche e ben definite.

3.5.2 Confronto tramite PCA

La proiezione PCA conferma le differenze tra i due approcci.

Nel caso **senza DBSCAN**, la presenza di rumore genera:

- una maggiore sovrapposizione visiva tra i cluster;
- una struttura meno compatta nello spazio delle componenti principali.

Dopo il filtraggio DBSCAN:

- la distribuzione dei punti appare più concentrata;
- le bande associate ai cluster risultano più definite.

In entrambi i casi, l’analisi dei *loadings* mostra che la separazione dei cluster è guidata principalmente da:

- tipologia di reato;
- contesto *Inside/Outside*;
- tipologia di arma;
- fascia oraria.

3.5.3 Heatmap vs Lift

La heatmap dei profili percentuali fornisce una prima descrizione dei cluster, ma evidenzia soprattutto:

- le feature più frequenti a livello globale;
- pattern dominanti ma non necessariamente distintivi.

Questa limitazione è particolarmente evidente nel clustering senza filtraggio, dove:

- le feature rare tendono a scomparire;
- i cluster appaiono simili tra loro nelle categorie più comuni.

L'analisi del **Lift** permette invece di:

- confrontare ogni cluster con la media globale;
- identificare le feature realmente sovra-rappresentate;
- assegnare una “personalità” a ciascun cluster.

Nel dataset filtrato, il Lift risulta particolarmente efficace nel mettere in evidenza:

- cluster dominati da rapine armate;
- cluster caratterizzati da aggressioni e reati notturni;
- cluster associati a crimini interni o a specifiche tipologie di arma.

3.5.4 Considerazioni conclusive

Il confronto tra i due approcci mostra che:

- il clustering senza DBSCAN offre una visione globale del fenomeno, ma risente maggiormente del rumore;
- il clustering con DBSCAN produce cluster più compatti, stabili e semanticamente interpretabili;
- l'analisi del Lift è fondamentale per comprendere le reali differenze tra i cluster, indipendentemente dal metodo di filtraggio; infatti, in entrambe le analisi abbiamo ottenuto 2 cluster con caratteristiche molto simili tra loro.

Nel complesso, l'integrazione di DBSCAN come fase di pre-filtraggio migliora la qualità interpretativa del clustering K-Means, senza alterare le principali strutture latenti del dataset, ma rendendole più evidenti e coerenti.

4 Timeseries

L'obiettivo dell'analisi è quello di esplorare, pulire e comprendere l'andamento temporale e spaziale dei crimini, applicando tecniche di **analisi di serie temporali e forecasting**, per prevedere l'evoluzione dei fenomeni criminali in determinati distretti o periodi.

Per l'analisi verranno utilizzate librerie **Python** come **pandas**, **matplotlib**, **seaborn**, **statsmodels**, **numpy**, **pmdarima** e **sklearn**, che consentiranno di effettuare una **pulizia accurata dei dati**, un' **analisi esplorativa** e la **modellazione del modello di clustering**.

4.1 Preparazione dati

Il processo ETL per l'analisi di forecasting viene approfondito attraverso una **aggregazione** per periodo per studiare l'evoluzione dei crimini nel tempo. Questa operazione consente di generare una serie temporale settimanale.

```
1 crime_time_series = (  
2 crime_data  
3 .set_index('DateTime')  
4 .resample('W')['Total Incidents']  
5 .sum()  
6 .reset_index())
```

Il risultato finale ottenuto tramite le operazioni di pulizia preliminari, è il seguente.

```
1 --- Valori mancanti dopo il fill ---  
2 Location 0  
3 Inside/Outside 0  
4 Weapon 0  
5 District 0  
6 Neighborhood 0  
7 dtype: int64  
8  
9 === Dimensione dataset pulito ===  
10 Serie temporale settimanale: (307, 2)  
11  
12 === Esempio serie temporale settimanale ===  
13 DateTime Total Incidents  
14 2011-01-02 281  
15 2011-01-09 800  
16 2011-01-16 750
```

```

17 2011-01-23 804
18 2011-01-30 667
19
20 === Statistiche descrittive della serie temporale settimanale ===
21         DateTime Total Incidents
22 count 307 307.000000
23 mean 2013-12-08 00:00:00 917.247557
24 min 2011-01-02 00:00:00 281.000000
25 25% 2012-06-20 12:00:00 851.000000
26 50% 2013-12-08 00:00:00 932.000000
27 75% 2015-05-27 12:00:00 1003.000000
28 max 2016-11-13 00:00:00 1114.000000
29 std NaN 116.174862

```

Listing 3: Output del processo di pulizia e aggregazione settimanale

4.2 Analisi esplorativa del dataset

Si effettuerà una prima analisi visiva del dataset graficando l'andamento della serie temporale dei crimini a Baltimora, in modo tale da poter avanzare una prima ipotesi sulla stazionarietà della serie.

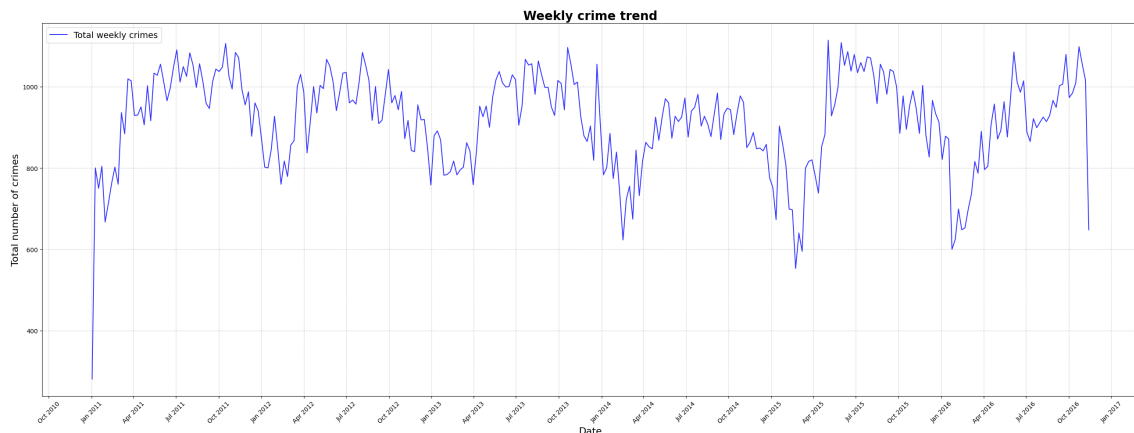


Figura 23: Weekly Crime Trend

Osservando il grafico (Figura 23), si possono notare alcune caratteristiche che suggeriscono che la serie **non è stazionaria**:

1. **Media non costante nel tempo**: ci sono periodi in cui il numero totale di incidenti settimanali è mediamente più alto (ad esempio tra il 2011 e il 2013, di nuovo intorno al 2015-2016). Altri periodi mostrano una diminuzione visibile (ad esempio inizio 2015 e inizio 2016). Questo indica che la **media varia nel tempo**.

2. **Varianza non costante nel tempo:** l'ampiezza delle fluttuazioni ("dispersione" dei valori) non è omogenea: in certi intervalli la variabilità sembra più elevata, mentre in altri le oscillazioni sono più contenute. Anche la **varianza**, quindi, sembra **cambiare nel tempo**.
3. **Possibile stagionalità:** il grafico mostra ciclicità annuale o semestrale: periodi ricorrenti in cui i crimini aumentano o diminuiscono, compatibili con **pattern stagionali** (ad esempio aumenti estivi o cali invernali). Questo tipo di oscillazione periodica è tipico di una serie **non stazionaria**, con componente stagionale.
4. **Valori più contenuti in certi intervalli:** si notano cali marcati in corrispondenza di alcuni anni (ad esempio inizio 2015 e 2016), forse legati a fattori specifici (politiche di sicurezza, eventi eccezionali, cambiamenti nei dati).

In sintesi, la serie sembra **non essere stazionaria**, poiché **media e varianza variano nel tempo** e sono presenti **elementi di stagionalità**. Per analisi predittive è necessario **stazionarla** tramite differenziazione o rimozione della stagionalità. Per effettuare un'analisi più approfondita, si decomporrà la serie nelle sue componenti stagionali, di trend e residuali.

```
1 decomposition = seasonal_decompose(  
2     crime_time_series.set_index('DateTime')['Total Incidents'],  
3     model='additive',  
4     period=52  
5 )
```

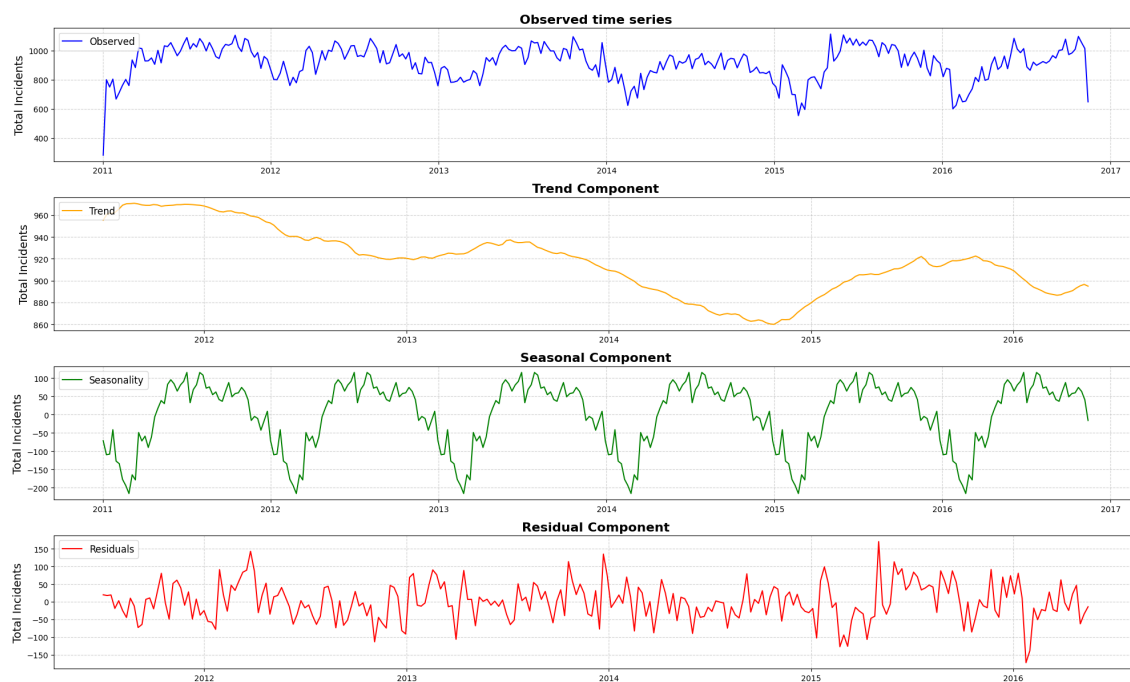



Figura 24: Decomposizione della serie

Analizzando il grafico della decomposizione della serie temporale (Figura 24), si osservano:

1. **Componente di Trend:** il trend mostra **una diminuzione graduale dal 2011 al 2014**, seguita da **una lieve risalita tra il 2015 e il 2016**. Questo suggerisce che, nel lungo periodo, il **numero medio di crimini settimanali tende a calare leggermente**, ma con una ripresa negli ultimi anni del campione. Quindi, **la media non è costante nel tempo**, confermando che la serie non è stazionaria in livello.
2. **Componente stagionale:** si osserva una **chiara periodicità annuale**, con oscillazioni regolari e simmetriche intorno allo zero. La stagionalità è **molto marcata e stabile nel tempo**, indicando che i crimini tendono ad aumentare e diminuire in determinati periodi dell'anno (ad esempio più incidenti nei mesi estivi, meno in quelli invernali). Questa regolarità suggerisce una **stagionalità fissa e ripetitiva**, il che è tipico di fenomeni sociali o ambienti influenzati da condizioni ricorrenti.
3. **Componente residuale:** i residui oscillano intorno allo zero senza una struttura evidente, ma **non appaiono completamente bianchi**: in certi periodi (ad esempio 2015) sembrano più dispersi o con outlier. Ciò indica che **parte della varianza non è spiegata** né dal trend né dalla stagionalità, ma non

emergono pattern sistematici forti. La varianza dei residui appare **piuttosto costante nel tempo**, un buon segno per modellazione successiva.

In sintesi, la serie **non è stazionaria in livello**, perché presenta **trend e stagionalità**. Dopo la rimozione del trend e della stagionalità o una differenziazione stagionale, i **residui potrebbero risultare stazionari** e quindi adatti a modelli successivi. Per confermare l'ipotesi, si effettua l'**ADF Test (Augmented Dickey-Fuller)** e si misura **p-value**, che per serie stazionarie dovrebbe essere inferiore alla soglia di *0.05*.

```
1 # Test di stazionariet ADF
2 adf_test = adfuller(crime_time_series['Total Incidents'])
3
4 adf_results = pd.Series({"ADF Test Statistic": adf_test[0],
5                          "p-value": adf_test[1],
6                          "Used Lag": adf_test[2],
7                          "Number of Observations": adf_test[3],
8                          "Critical Value (1%)": adf_test[4]['1%'],
9                          "Critical Value (5%)": adf_test[4]['5%'],
10                         "Critical Value (10%)": adf_test[4]['10%']})
11
12 print("\n=== Risultati del test di stazionariet ADF ===")
13 adf_results
```

```
1 ADF Test Statistic -4.303567
2 p-value 0.000438
3 Used Lag 9.000000
4 Number of Observations 297.000000
5 Critical Value (1%) -3.452561
6 Critical Value (5%) -2.871321
7 Critical Value (10%) -2.571982
8 dtype: float64
```

Listing 4: Risultati test ADF

In contrapposizione all'analisi effettuata, osservando i valori dell'ADF Test si può **rifiutare l'ipotesi nulla (H0)** con elevata confidenza (livello superiore al 99%) e concludere che la serie è **stazionaria**. Tuttavia, ci sono alcune considerazioni da tenere a mente:

- **Statisticamente**, l'ADF dice che la serie può essere trattata come stazionaria.
- **Visivamente**, però, rimangono segnali di trend e stagionalità.

Si può quindi procedere con l'analisi dei **Correlogrammi ACF e PACF** per stimare i parametri del modello SARIMA.

```
1 # Creazione dei grafici ACF e PACF
2 fig, axes = plt.subplots(1, 2, figsize=(20, 6), dpi = 100)
3
4 # Grafico ACF
5 plot_acf(crime_time_series['Total Incidents'], lags=52, ax=axes[0])
6 axes[0].set_title('Autocorrelation Function (ACF)', fontsize=16,
7                  fontweight='bold')
8 axes[0].set_xlabel('Lags', fontsize=14)
9 axes[0].set_ylabel('Autocorrelation', fontsize=14)
10 axes[0].grid(True, linestyle='--', alpha=0.6)
11
12 # Grafico PACF
13 plot_pacf(crime_time_series['Total Incidents'], lags=52, ax=axes[1])
14 axes[1].set_title('Partial Autocorrelation Function (PACF)', fontsize
15                  =16, fontweight='bold')
16 axes[1].set_xlabel('Lags', fontsize=14)
17 axes[1].set_ylabel('Partial Autocorrelation', fontsize=14)
18 axes[1].grid(True, linestyle='--', alpha=0.6)
19
20 plt.tight_layout()
21 plt.show()
```

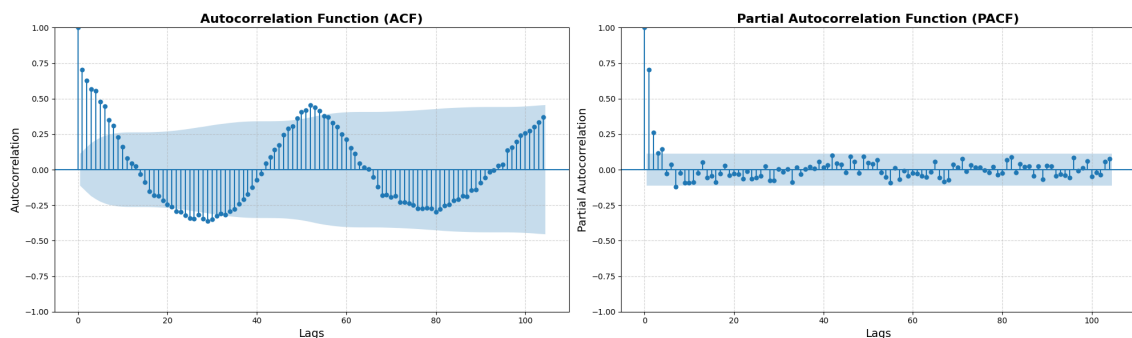


Figura 25: Autocorrelogrammi della serie originale

Nonostante il test ADF indichi che la serie è stazionaria, i grafici mostrano che **c'è ancora una struttura molto forte** che deve essere modellata.

- **ACF**: l'ACF mostra la correlazione della serie con se stessa a vari ritardi (lags). La caratteristica più evidente è un pattern sinusoidale, che indica una

forte stagionalità. Dato che il picco si ripete ogni ~ 52 lag, questo grafico ci dice che i crimini a Baltimora seguono un ciclo annuale molto preciso.

- **PACF**: isola la correlazione diretta tra un lag e l'altro, rimuovendo gli effetti intermedi. C'è un taglio netto già dai primi lag: il Lag 1 è molto alto (0.7), il Lag 2 è positivo ma molto più basso (circa 0.25). Dal Lag 3 in poi, la maggior parte dei punti rientra nell'area azzurra (l'intervallo di confidenza), il che significa che non sono statisticamente significativi (o sono rumore bianco).
- **Interpretazione**: Quando l'ACF decade lentamente (o in questo caso ondulata) e la PACF si interrompe bruscamente dopo pochi lag, potrebbe essere il segnale di un processo Autoregressivo (AR), probabilmente un termine **AR(1)** o **AR(2)**.

Quindi, per quanto riguarda la **componente non stagionale** (p, d, q), probabilmente avremo dei valori:

- p (**AR**): probabilmente 1 o 2, visto il taglio netto nella PACF dopo il lag 2;
- d : 0, poiché il test ADF ha detto che è stazionaria;
- q (**MA**): probabilmente 0, poiché l'ACF non ha un taglio netto ma un decadimento.

Per quanto riguarda la **componente stagionale** (P, D, Q):

- D : anche se ADF dice che è stazionaria, il pattern ad onda è molto forte, quindi potrebbe essere utile provare una differenziazione stagionale.

In conclusione, prima di effettuare una differenziazione, si tenterà comunque di osservare cosa succede con la serie originale.

Verrà eseguito **Auto Arima** sulla serie originale per trovare i valori dei parametri ottimali.

```
1 # Eseguiamo auto_arima specificando il periodo stagionale 'm'
2 sarima_model = pm.auto_arima(
3     crime_time_series['Total Incidents'], # La serie temporale
4     seasonal=True, # Abilita la ricerca di un modello stagionale
5     m=52, # Il periodo a 52 settimane
6
7     # Parametri per guidare la ricerca
8     d=None, # Non forza una differenziazione semplice
```

```

9      D=None, # Non forza una differenziazione stagionale
10     test='adf', # Usa il test ADF per determinare d
11     start_p=0, start_q=0, # Punto di partenza per i parametri non
        stagionali
12     start_P=0, start_Q=0, # Punto di partenza per i parametri
        stagionali
13     max_p=2, max_q=2, # Limite massimo per p e q
14     max_P=2, max_Q=2, # Limite massimo per P e Q
15
16     # Parametri per il controllo del processo
17     trace=True, # Mostra i modelli che sta testando
18     error_action='ignore', # Ignora i modelli che non convergono
19     suppress_warnings=True, # Sopprime i warning
20     stepwise=True # Usa un approccio efficiente per la ricerca
21 )
22
23 # Stampa il sommario del modello migliore trovato
24 print(sarima_model.summary())

```

```

1 Performing stepwise search to minimize aic
2 ARIMA(0,0,0)(0,0,0)[52] intercept : AIC=3793.856, Time=0.03 sec
3 ARIMA(1,0,0)(1,0,0)[52] intercept : AIC=3540.906, Time=6.25 sec
4 ARIMA(0,0,1)(0,0,1)[52] intercept : AIC=3633.306, Time=2.90 sec
5 ARIMA(0,0,0)(0,0,0)[52] : AIC=5066.424, Time=0.00 sec
6 ARIMA(1,0,0)(0,0,0)[52] intercept : AIC=3544.210, Time=0.06 sec
7 ARIMA(1,0,0)(2,0,0)[52] intercept : AIC=3536.478, Time=70.68 sec
8 ARIMA(1,0,0)(2,0,1)[52] intercept : AIC=3541.453, Time=86.73 sec
9 ARIMA(1,0,0)(1,0,1)[52] intercept : AIC=3545.531, Time=5.19 sec
10 ARIMA(0,0,0)(2,0,0)[52] intercept : AIC=3785.813, Time=104.27 sec
11 ARIMA(2,0,0)(2,0,0)[52] intercept : AIC=3527.385, Time=88.68 sec
12 ARIMA(2,0,0)(1,0,0)[52] intercept : AIC=3531.325, Time=6.78 sec
13 ARIMA(2,0,0)(2,0,1)[52] intercept : AIC=3528.778, Time=107.52 sec
14 ARIMA(2,0,0)(1,0,1)[52] intercept : AIC=3534.167, Time=7.05 sec
15 ARIMA(2,0,1)(2,0,0)[52] intercept : AIC=3528.279, Time=115.83 sec
16 ARIMA(1,0,1)(2,0,0)[52] intercept : AIC=3530.258, Time=101.55 sec
17 ARIMA(2,0,0)(2,0,0)[52] : AIC=inf, Time=50.56 sec
18
19 Best model: ARIMA(2,0,0)(2,0,0)[52] intercept
20 Total fit time: 754.160 seconds
21
22 SARIMAX Results
=====
23
    ↪
23 Dep. Variable: y No. Observations: 307

```

```

24 Model: SARIMAX(2, 0, 0)x(2, 0, 0, 52) Log Likelihood -1757.692
25 Date: Thu, 29 Jan 2026 AIC 3527.385
26 Time: 16:58:27 BIC 3549.746
27 Sample: 0 HQIC 3536.327
28                                     - 307
29 Covariance Type: opg
30 =====
31      ↪
32      coef std err z P>|z| [0.025 0.975]
33 -----
34      ↪
35 intercept 171.8439 29.008 5.924 0.000 114.990 228.698
36 ar.L1 0.5260 0.051 10.219 0.000 0.425 0.627
37 ar.L2 0.2008 0.066 3.021 0.003 0.071 0.331
38 ar.S.L52 0.1301 0.065 2.015 0.044 0.004 0.257
39 ar.S.L104 0.1810 0.089 2.027 0.043 0.006 0.356
40 sigma2 5411.5600 306.951 17.630 0.000 4809.946 6013.174
41 =====
42      ↪
43 Ljung-Box (L1) (Q): 1.34 Jarque-Bera (JB): 568.77
44 Prob(Q): 0.25 Prob(JB): 0.00
45 Heteroskedasticity (H): 1.02 Skew: -0.95
46 Prob(H) (two-sided): 0.91 Kurtosis: 9.39
47 =====
48      ↪
49 Warnings:
50 [1] Covariance matrix calculated using the outer product of gradients (
51      ↪ complex-step).

```

Listing 5: Risultati Auto Arima Serie Originale

Come si può osservare dal Listato 5, il modello migliore trovato è quello con parametri $p=2$, $d=0$, $q=0$, $P=2$, $D=0$, $Q=0$.

- **Conferma della Stazionarietà:**

- **$d=0$:** il modello non ha applicato differenziazioni e la serie originale è stata accettata come stazionaria (grazie al test ADF).
- **$D=0$:** anche la componente stagionale è stabile e non richiede differenziazione.

- **Struttura del Modello:** il modello è puramente **Autoregressivo (AR)**, sia a breve termine sia nella sua stagionalità.

- **AR(2) (Semplice)**: i coefficienti *ar.L1* e *ar.L2* sono significativi. Ciò significa che il numero di crimini della settimana corrente dipende fortemente dai crimini di **1 e 2 settimane fa**.
- **Seasonal AR(2) (Stagionale)**: i coefficienti *ar.S.L52* e *ar.S.L104* sono significativi, quindi il modello guarda cos'è successo esattamente **1 e 2 anni fa (52-104 settimane)** per fare la previsione corrente.

Questo ha senso per i dati sul crimine: se la criminalità sale a breve termine, tende a restare alta per un po'. Inoltre, c'è una forte ricorrenza annuale (ad esempio crimini estivi vs invernali).

- **Diagnostica dei residui:**

- **Ljung-Box (Prob(Q) = 0.25)**: si accetta l'ipotesi nulla, quindi i residui sono scorrelati (rumore bianco). Il modello ha catturato tutto il segnale temporale possibile.
- **Jarque-Bera (Prob(JB) = 0.00)**: il p-value è minore di 0.05, mostrando che i residui non seguono una distribuzione normale. La **Kurtosis** alta (9.39) suggerisce che la distribuzione degli errori ha "code più pesanti", ovvero ci sono più valori estremi (outlier) di quanto ci si aspetterebbe. Sebbene non sia ideale, questo è molto comune nelle serie storiche reali e, per scopi di previsione, è meno grave della presenza di autocorrelazione.

Si procede con la diagnostica e l'analisi dei residui prodotti da tale modello, per assicurarci che non ci siano pattern particolari, cercando media e varianza costante.

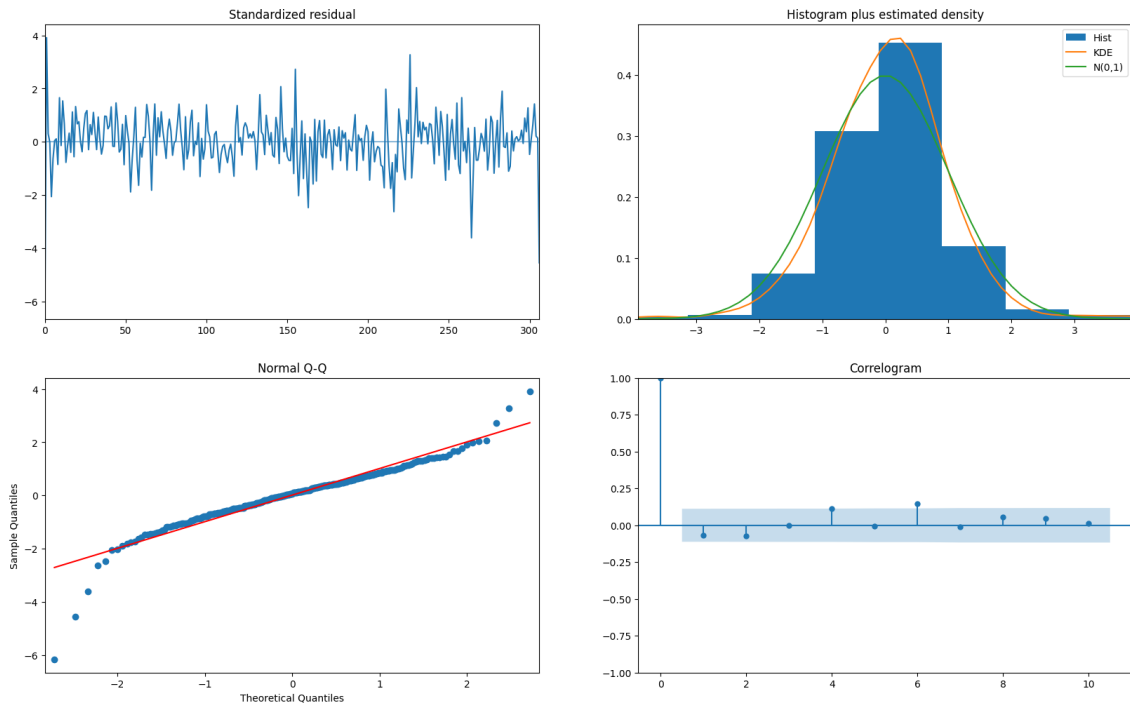


Figura 26: Diagnostica e analisi dei residui per la serie originale

Osservando i grafici della Figura 26, si può dire che:

1. Standardized Residuals

Questo grafico mostra gli errori del modello nel tempo, standardizzati in modo da avere una deviazione standard di 1.

- I residui fluttuano in modo casuale attorno alla linea dello zero. Non c'è alcun trend, ciclo o pattern evidente. La varianza appare costante.
- Concludendo, questo è l'aspetto ideale del “rumore bianco”, quindi il modello **ha estratto con successo tutta l'informazione prevedibile** dalla serie storica, lasciando solo un errore casuale e imprevedibile.

2. Histogram plus estimated density

Questo grafico confronta la distribuzione dei residui (istogramma blu e curva KDE arancione) con una distribuzione normale perfetta (curva verde $N(0,1)$).

- La curva arancione (KDE) è centrata sullo zero e ha una forma a campana, ma è leggermente più “appuntita” e alta rispetto alla curva normale verde ed è leggermente spostata verso destra.
- Visivamente viene confermato il risultato del test di Jarque-Bera. I residui **non sono perfettamente normali**, ma sono molto vicini. La

forma più appuntita (leptocurtica) è comune nei dati reali e non invalida il modello per scopi di previsione.

3. Normal Q-Q Plot

Questo grafico confronta i quantili dei residui (punti blu) con i quantili di una distribuzione normale teorica (linea rossa).

- La stragrande maggioranza dei punti si allinea quasi perfettamente lungo la linea rossa. Si notano solo leggere deviazioni alle estremità, che corrispondono agli outlier.
- Questo è un risultato eccellente e rafforza la conclusione precedente. Nonostante la non-normalità statistica, la distribuzione **si avvicina molto a una distribuzione normale** per la maggior parte dei valori.

4. Correlogram (ACF dei Residui)

Questo grafico mostra se gli errori del modello sono correlati tra loro a diversi lag.

- **Nessuna** delle barre di autocorrelazione (a eccezione del lag 0, che è sempre 1) **supera in modo significativo l'area di confidenza blu**.
- Questo conferma visivamente il test di Ljung-Box ($\text{Prob}(Q) > 0.05$): quindi **non c'è alcuna autocorrelazione residua significativa**. Il modello ha catturato tutta la dipendenza temporale (sia a breve termine sia stagionale) presente nei dati.

4.3 Addestramento del modello sulla serie originale

Si addestrerà il modello dividendo il dataset in 80% training e 20% testing, così da poter simulare un addestramento sugli anni di misurazione (7) e valutare i risultati su un intero anno.

```
1 # Divisione in training e test set (80% training, 20% test)
2 train_size = int(len(crime_time_series) * 0.8)
3 train, test = crime_time_series[:train_size], crime_time_series[
    train_size:]
4
5 # Addestramento del modello SARIMA con i parametri trovati
6 model = SARIMAX(train['Total Incidents'],
7                 order=(2, 0, 0),
8                 seasonal_order=(2, 0, 0, 52),
9                 trend='c')
```

```

10 model_fit = model.fit(dispatch=False)
11
12 # Previsioni sul test set
13 forecast_object = model_fit.get_forecast(steps=len(test))
14 forecast_mean = forecast_object.predicted_mean
15 forecast_ci = forecast_object.conf_int()
16
17 # Calcolo degli errori
18 mse = mean_squared_error(test['Total Incidents'], forecast_mean)
19 mae = mean_absolute_error(test['Total Incidents'], forecast_mean)
20 rmse = np.sqrt(mse)

```

```

1 Mean Absolute Error (MAE): 75.32
2 Root Mean Squared Error (RMSE): 100.55

```

Listing 6: Valori degli Errori

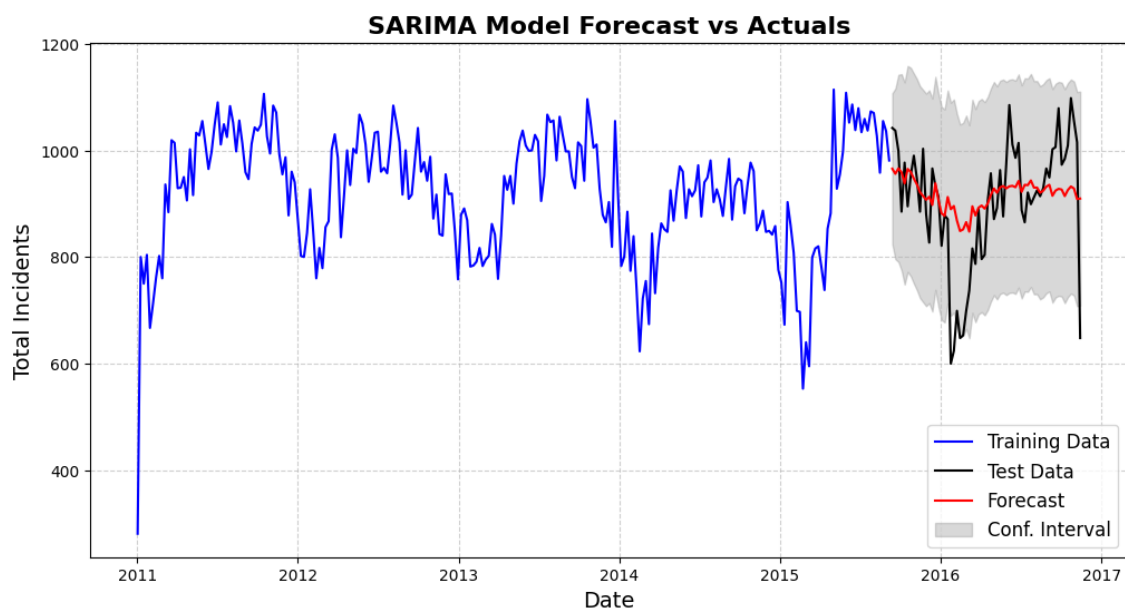


Figura 27: Modello SARIMAX serie originale

Analizzando il grafico del modello SARIMAX (Figura 27), si possono effettuare le seguenti considerazioni:

- **Previsione (Linea rossa):** il modello riesce a individuare il **trend medio** e una vaga ciclicità stagionale, ma **fallisce nel catturare l'ampiezza dei picchi e delle valli**. La linea rossa appare "smorzata" rispetto alla realtà (linea nera), indicando che il modello tende a essere troppo conservativo. In pratica, sottostima i periodi di alta criminalità e sovrastima quelli di bassa.

- **Dati Reali (Linea nera):** la varianza dei dati reali è molto più elevata di quanto previsto dal modello. Ci sono picchi improvvisi che la semplice storia passata non è sufficiente a spiegare.
- **Intervallo di confidenza (Area grigia):** sebbene i dati reali cadano spesso nell'intervallo, quest'ultimo è piuttosto ampio, segnale che il modello ha una certezza limitata.

Le metriche d'errore (Listato 10), inoltre, confermano l'impressione del grafico.

- **MAE (Mean Absolute Error - 75.32 e RMSE (Root Mean Squared Error) - 100.55:** l'elevato divario tra i valori conferma che ci sono **grandi errori puntuali** (gli outlier dei picchi che il modello non vede).
- **Conclusione preliminare:** un modello puramente statistico (SARIMA) fornisce una **baseline accettabile, ma non sufficiente**. Riesce a dirci "più o meno" quanti crimini avverranno, ma non riesce a modellare la complessità e la variabilità settimana per settimana.

Sebbene l'algoritmo e i test di stazionarietà abbiano suggerito un modello senza differenziazioni, l'analisi visiva mostra una previsione eccessivamente conservativa, che tende a ritornare verso la media storica, fallendo nel raggiungere i picchi stagionali reali.

Per migliorare la capacità del modello di catturare l'ampiezza delle oscillazioni, una strategia efficace potrebbe essere **forzare manualmente una differenziazione stagionale**, anche andando contro i test preliminari. In questo scenario, il modello non cercherebbe più di prevedere il numero assoluto di crimini, ma la **differenza rispetto alla stessa settimana dell'anno precedente**.

Questo approccio può comportare due potenziali benefici:

- **Maggiore Reattività:** costringe il modello a "replicare la forma" dell'anno precedente, permettendogli di seguire meglio i picchi e le valli stagionali anziché appiattirsi.
- **Gestione di Trend Locali:** aiuta a catturare eventuali cambiamenti nel livello base della criminalità che i coefficienti AR puri potrebbero perdere su lunghi orizzonti.

4.4 Analisi della serie differenziata

A questo punto, si procederà l'analisi effettuando due approcci di differenziazione:

- **Semplice:** per rimuovere il trend, con l'effetto di trasformare la serie che oscilla attorno a una media costante, di fatto "appiattendolo" il trend.
- **Stagionale:** per rimuovere pattern annuali o periodici, eliminando la dipendenza ciclica annuale; confrontando ogni settimana con la stessa settimana dell'anno prima, si annulla l'effetto sistematico legato a quel particolare periodo dell'anno.

```
1 # Differenziazione semplice
2 crime_diff = crime_time_series['Total Incidents'].diff().dropna()
3
4 # Differenziazione stagionale (period=52 settimane)
5 crime_seasonal_diff = crime_time_series['Total Incidents'].diff(52).
  dropna()
```



Figura 28: Differenziazioni semplice e stagionale

```

1  === Risultati del test di stazionariet ADF sulla serie differenziata semplice
    ↪ ===
2  ADF Test Statistic -7.476872e+00
3  p-value 4.887433e-11
4  Used Lag 5.000000e+00
5  Number of Observations 3.000000e+02
6  Critical Value (1%) -3.452337e+00
7  Critical Value (5%) -2.871223e+00
8  Critical Value (10%) -2.571929e+00
9  dtype: float64
10
11 === Risultati del test di stazionariet ADF sulla serie differenziata
    ↪ stagionale ===
12 ADF Test Statistic -4.166292
13 p-value 0.000751
14 Used Lag 5.000000
15 Number of Observations 249.000000
16 Critical Value (1%) -3.456888
17 Critical Value (5%) -2.873219
18 Critical Value (10%) -2.572994
19 dtype: float64

```

Listing 7: Risultati test ADF per le differenziazioni

Osservando i grafici (Figura 28) e i risultati dei test ADF (Listato 7), si può concludere che:

1. **Differenziazione semplice:** la statistica ADF (-7.48) è **molto più piccola** del valore critico al 1% (-3.45). Il p-value è **praticamente zero**. Conclusione: la serie differenziata semplice è **fortemente stazionaria**, il trend è stato completamente rimosso e ora la media e la varianza sono costanti nel tempo.
2. **Differenziazione stagionale (lag = 52 settimane):** anche qui la statistica ADF è **inferiore** al valore critico al 1% (-3.46). Il p-value è ancora **molto basso** (<0.01). Conclusione: la serie differenziata stagionale è anch'essa **stazionaria**, ma l'effetto è **meno marcato** rispetto alla differenziazione semplice.

In sintesi, la **differenziazione semplice** ha reso la serie fortemente stazionaria eliminando il trend principale, mentre la **differenziazione stagionale** ha rimosso le oscillazioni annuali, ma non riduce tanto la dipendenza come prima. Entrambe le serie, quindi, risultano **statisticamente stazionarie**, ma per motivi diversi.

Per modelli come **SARIMA**, dove è utile catturare sia trend che stagionalità, una soluzione potrebbe essere **applicare entrambe le differenziazioni**, verificando di nuovo la stazionarietà e la stabilità visiva, dato che, in genere, la doppia differenziazione fornisce una serie finale stazionaria in media e varianza, pronta per il fitting di un modello ARIMA/SARIMA.

```
1 # Differenziazione completa (semplice + stagionale)
2 crime_diff_full = crime_time_series['Total Incidents'].diff().diff(52)
   .dropna()
```

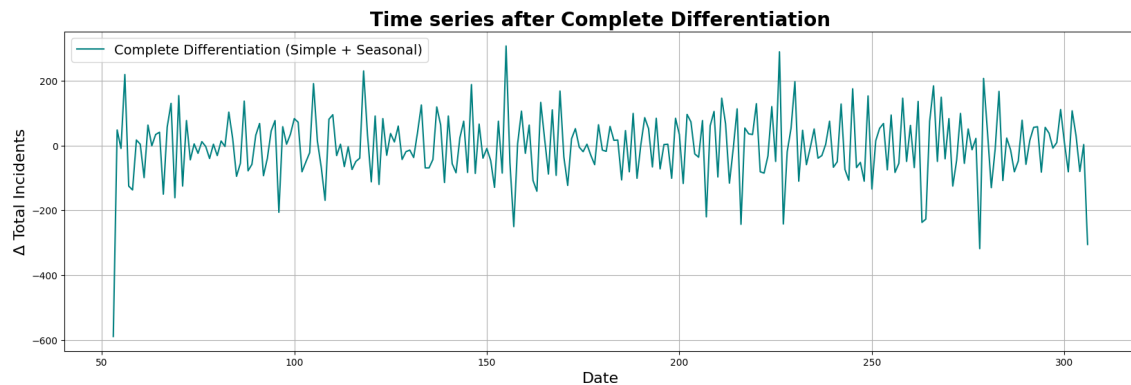


Figura 29: Differenziazione totale

Nel grafico (Figura 29) possiamo osservare che la serie differenziata completa (semplice + stagionale) oscilla chiaramente **intorno allo zero**, senza trend a lungo termine evidente. Inoltre, la varianza è **piuttosto costante**, con un'ampiezza delle oscillazioni relativamente consistenti lungo tutto l'asso temporale. Infine, non si osservano più pattern stagionali regolari.

```
1 === Risultati del test di stazionariet ADF sulla serie completamente
   ↪ differenziata ===
2 ADF Test Statistic -1.157592e+01
3 p-value 3.022940e-21
4 Used Lag 4.000000e+00
5 Number of Observations 2.490000e+02
6 Critical Value (1%) -3.456888e+00
7 Critical Value (5%) -2.873219e+00
8 Critical Value (10%) -2.572994e+00
9 dtype: float64
```

Listing 8: Risultati test ADF per differenziazione totale

Osservando i risultati del test di stazionarietà ADF applicato alla serie differenziata completa (Listato 8), si può concludere che la serie è **pienamente stazionaria**.

Si può quindi procedere con l'analisi dei **correlogrammi ACF e PACF** per stimare i parametri (p, d, q) del modello SARIMA.

```
1 # Grafico ACF
2 plot_acf(crime_diff_full, lags=52, ax=axes[0])
3 axes[0].set_title('Autocorrelation Function (ACF)', fontsize=16,
4                   fontweight='bold')
5 axes[0].set_xlabel('Lags', fontsize=14)
6 axes[0].set_ylabel('Autocorrelation', fontsize=14)
7 axes[0].grid(True, linestyle='--', alpha=0.6)
8
9 # Grafico PACF
10 plot_pacf(crime_diff_full, lags=52, ax=axes[1])
11 axes[1].set_title('Partial Autocorrelation Function (PACF)', fontsize
12                  =16, fontweight='bold')
13 axes[1].set_xlabel('Lags', fontsize=14)
14 axes[1].set_ylabel('Partial Autocorrelation', fontsize=14)
15 axes[1].grid(True, linestyle='--', alpha=0.6)
```

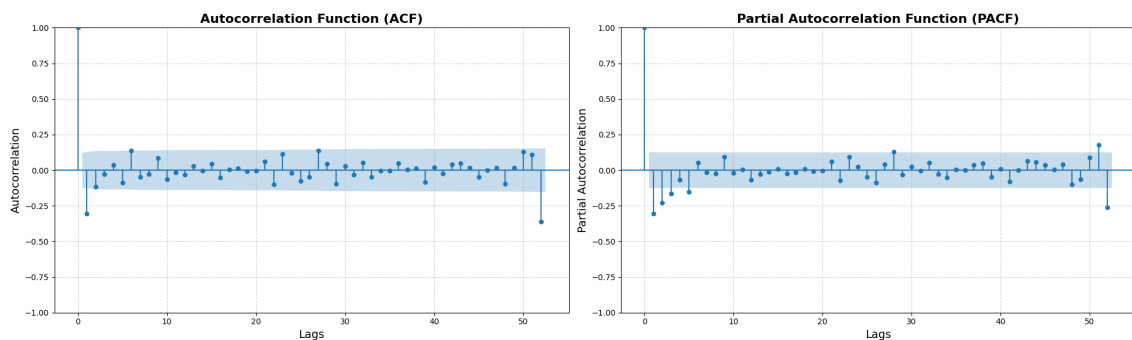


Figura 30: Autocorrelogrammi

Per identificare i parametri **p (ordine AR)** e **q (ordine MA)**, si osservano i primi lag dei grafici:

- **Grafico ACF:** c'è un picco negativo statisticamente significativo al **lag 1**. Subito dopo, i valori successivi cadono rapidamente all'interno della banda di confidenza (area azzurra). Questo pattern con un taglio netto e improvviso dopo il primo lag, è caratteristico di un **modello a Media Mobile (MA)**. L'ordine del modello è dato dall'ultimo lag significativo prima del taglio, suggerendo quindi un parametro **q = 1**.
- **Grafico PACF:** anche qui c'è un picco negativo significativo al **lag 1**, e i valori successivi tendono a cadere rapidamente all'interno della banda di confidenza.

Un taglio netto nel PACF è tipico di un **modello Autoregressivo (AR)**, suggerendo un parametro $p = 1$.

Avendo indicazioni sia per un modello AR(1) sia per un MA(1) e analizzando entrambi i grafici, si osserva che il grafico ACF è molto netto e pulito, rendendo l'ipotesi di un **modello MA(1) ($q = 1, p = 0$)** leggermente più probabile e un ottimo punto di partenza.

Per determinare i parametri stagionali **P** e **Q**, si osservano i lag specificamente vicini al 52:

- **Grafico ACF**: si nota un picco negativo statisticamente significativo al **lag 52**. Questo picco è isolato, il che è un segnale molto forte per un modello di **Media Mobile Stagionale (SMA)**. Questo conferma la precedente ipotesi, suggerendo un valore di $Q = 1$.
- **Grafico PACF**: analogamente, si osserva un picco negativo significativo al **lag 52**. Anche questo picco sembra essere isolato, suggerendo un modello **Autoregressivo Stagionale (SAR)** e un valore di $P = 1$.

Verrà eseguito di nuovo **Auto Arima**, specificando il periodo stagionale e forzando la differenziazione per trovare il modello migliore.

```
1 sarima_model = pm.auto_arima(  
2     crime_time_series['Total Incidents'], # La serie temporale  
3     seasonal=True, # Abilita la ricerca di un modello stagionale  
4     m=52, # Il periodo a 52 settimane  
5  
6     # Parametri per guidare la ricerca  
7     d=1, # Forza una differenziazione semplice (per il trend)  
8     D=1, # Forza una differenziazione stagionale  
9     start_p=0, start_q=0, # Punto di partenza per i parametri non  
10         stagionali  
11     start_P=0, start_Q=0, # Punto di partenza per i parametri  
12         stagionali  
13  
14     max_p=2, max_q=2, # Limite massimo per p e q  
15     max_P=2, max_Q=2, # Limite massimo per P e Q  
16  
17     # Parametri per il controllo del processo  
18     trace=True, # Mostra i modelli che sta testando  
19     error_action='ignore', # Ignora i modelli che non convergono  
20     suppress_warnings=True, # Sopprime i warning
```



```

18     stepwise=True # Usa un approccio efficiente per la ricerca
19 )
20
21 # Stampa il sommario del modello migliore trovato
22 print(sarima_model.summary())

```

```

1 Performing stepwise search to minimize aic
2 ARIMA(0,1,0)(0,1,0)[52] : AIC=3084.654, Time=0.57 sec
3 ARIMA(1,1,0)(1,1,0)[52] : AIC=3001.796, Time=16.69 sec
4 ARIMA(0,1,1)(0,1,1)[52] : AIC=inf, Time=57.68 sec
5 ARIMA(1,1,0)(0,1,0)[52] : AIC=3057.527, Time=1.07 sec
6 ARIMA(1,1,0)(2,1,0)[52] : AIC=2989.641, Time=60.63 sec
7 ARIMA(1,1,0)(2,1,1)[52] : AIC=2991.259, Time=176.38 sec
8 ARIMA(1,1,0)(1,1,1)[52] : AIC=2990.250, Time=118.76 sec
9 ARIMA(0,1,0)(2,1,0)[52] : AIC=inf, Time=57.87 sec
10 ARIMA(2,1,0)(2,1,0)[52] : AIC=2974.791, Time=87.72 sec
11 ARIMA(2,1,0)(1,1,0)[52] : AIC=2988.008, Time=22.12 sec
12 ARIMA(2,1,0)(2,1,1)[52] : AIC=2976.337, Time=208.68 sec
13 ARIMA(2,1,0)(1,1,1)[52] : AIC=2975.967, Time=142.43 sec
14 ARIMA(2,1,1)(2,1,0)[52] : AIC=2960.771, Time=156.23 sec
15 ARIMA(2,1,1)(1,1,0)[52] : AIC=2966.596, Time=42.47 sec
16 ARIMA(2,1,1)(2,1,1)[52] : AIC=2962.539, Time=361.75 sec
17 ARIMA(2,1,1)(1,1,1)[52] : AIC=2961.167, Time=187.17 sec
18 ARIMA(1,1,1)(2,1,0)[52] : AIC=2958.777, Time=130.16 sec
19 ARIMA(1,1,1)(1,1,0)[52] : AIC=2964.613, Time=40.86 sec
20 ARIMA(1,1,1)(2,1,1)[52] : AIC=2960.546, Time=263.86 sec
21 ARIMA(1,1,1)(1,1,1)[52] : AIC=2959.168, Time=144.58 sec
22 ARIMA(0,1,1)(2,1,0)[52] : AIC=2960.797, Time=86.79 sec
23 ARIMA(1,1,2)(2,1,0)[52] : AIC=inf, Time=377.63 sec
24 ARIMA(0,1,2)(2,1,0)[52] : AIC=2958.837, Time=97.30 sec
25 ARIMA(2,1,2)(2,1,0)[52] : AIC=inf, Time=324.28 sec
26 ARIMA(1,1,1)(2,1,0)[52] intercept : AIC=2960.279, Time=146.04 sec

```

```

27
28 Best model: ARIMA(1,1,1)(2,1,0)[52]

```

```

29 Total fit time: 3309.859 seconds

```

```

30 SARIMAX Results

```

```

31 =====

```

```

32 Dep. Variable: y No. Observations: 307
33 Model: SARIMAX(1, 1, 1)x(2, 1, [], 52) Log Likelihood -1474.389
34 Date: Fri, 14 Nov 2025 AIC 2958.777
35 Time: 16:34:31 BIC 2976.464
36 Sample: 0 HQIC 2965.892

```

```

37 - 307

```

```

38 Covariance Type: opg

```

```

39 =====
40      ↪
41      coef std err z P>|z| [0.025 0.975]
42 -----
43      ↪
44 ar.L1 0.2262 0.079 2.854 0.004 0.071 0.382
45 ma.L1 -0.8219 0.051 -16.055 0.000 -0.922 -0.722
46 ar.S.L52 -0.6176 0.067 -9.163 0.000 -0.750 -0.485
47 ar.S.L104 -0.2403 0.064 -3.780 0.000 -0.365 -0.116
48 sigma2 5925.4936 387.760 15.281 0.000 5165.499 6685.488
49 =====
50      ↪
51 Ljung-Box (L1) (Q): 1.32 Jarque-Bera (JB): 199.54
52 Prob(Q): 0.25 Prob(JB): 0.00
53 Heteroskedasticity (H): 1.07 Skew: -0.66
54 Prob(H) (two-sided): 0.76 Kurtosis: 7.14
55 =====
56      ↪

```

Listing 9: Risultati Auto Arima

Come si può osservare dal Listato 9, il modello migliore trovato è quello con parametri $p=1$, $d=1$, $q=1$, $P=2$, $D=1$, $Q=0$.

- **Componente non stagionale:**

- **$d=1$:** conferma che è stata necessaria una differenziazione semplice per rimuovere il trend.
- **$p=1$:** il numero di incidenti di una settimana è influenzato dal numero di incidenti della settimana precedente.
- **$q=1$:** il modello viene corretto anche in base all'errore di previsione commesso nella settimana precedente.

- **Componente stagionale:**

- **$D=1$:** conferma che sia stata necessaria una differenziazione stagionale per rimuovere il ciclo annuale.
- **$P=2$:** il numero di incidenti di una settimana è influenzato non solo dal valore della stessa settimana dell'anno precedente (lag 52), ma anche da quello della stessa settimana di due anni prima (lag 104). Questo indica un pattern stagionale molto forte e persistente nel tempo.
- **$Q=0$:** il modello non ha trovato necessario includere un termine di media mobile stagionale.

- **Diagnostica dei residui:**

- **Ljung-Box ($\text{Prob}(Q) = 0.25$):** il p-value è maggiore di 0.05, indicando che i residui non sono autocorrelati. Il modello ha catturato con successo tutta la struttura temporale presente nei dati, lasciando solo "rumore bianco".
- **Heteroskedasticity ($\text{Prob}(H) = 0.76$):** il p-value è maggiore di 0.05, confermando che la varianza degli errori è costante nel tempo (omoschedasticità). Il modello è stabile e non ha periodi in cui commette errori sistematicamente più grandi o più piccoli.
- **Jarque-Bera ($\text{Prob}(JB) = 0.00$):** il p-value è minore di 0.05, mostrando che i residui non seguono una distribuzione normale, come per la serie originale.

Anche qui, si può procedere con la diagnostica e l'analisi dei residui prodotti da tale modello, per assicurarci che non ci siano pattern particolari, cercando media e varianza costante.

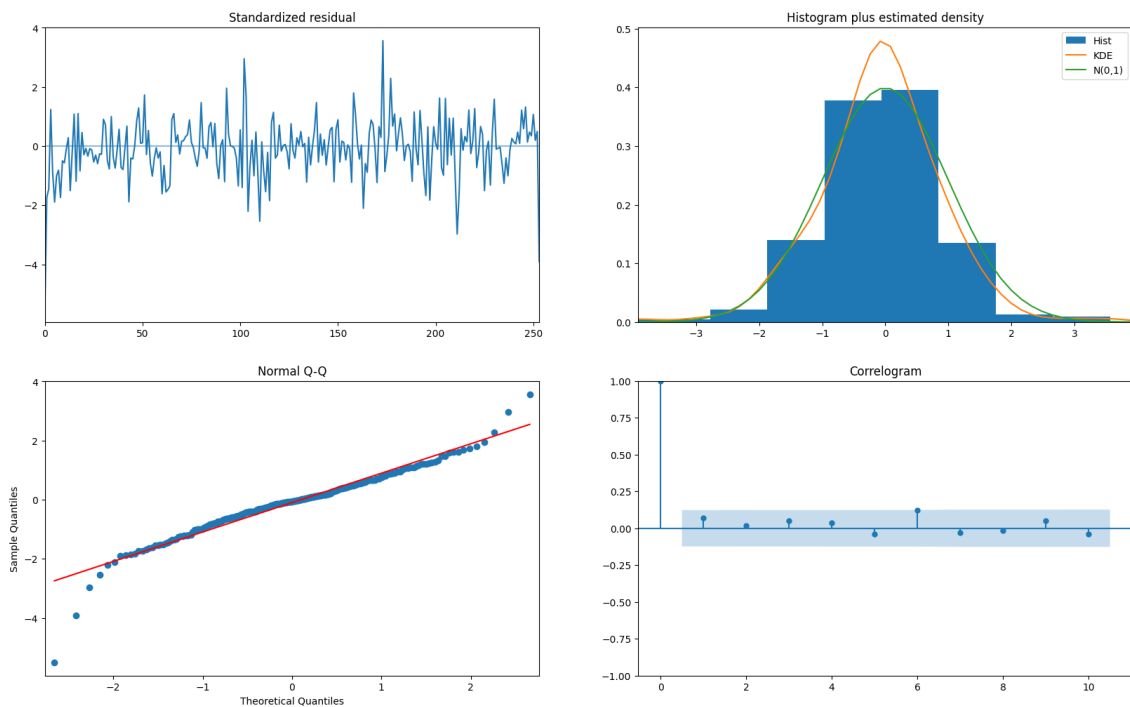


Figura 31: Diagnostica e analisi dei residui della serie differenziata

Come per la serie originale, osservando i grafici della Figura 31, si può dire che:

1. Standardized Residuals

Questo grafico mostra gli errori del modello nel tempo, standardizzati in modo da avere una deviazione standard di 1.

- I residui fluttuano in modo casuale attorno alla linea dello zero. Non c'è alcun trend, ciclo o pattern evidente. La varianza appare costante.
- Concludendo, questo è l'aspetto ideale del “rumore bianco”, quindi il modello **ha estratto con successo tutta l'informazione prevedibile** dalla serie storica, lasciando solo un errore casuale e imprevedibile.

2. Histogram plus estimated density

Questo grafico confronta la distribuzione dei residui (istogramma blu e curva KDE arancione) con una distribuzione normale perfetta (curva verde $N(0,1)$).

- La curva arancione (KDE) è centrata sullo zero e ha una forma a campana, ma è leggermente più “appuntita” e alta rispetto alla curva normale verde.
- Visivamente viene confermato il risultato del test di Jarque-Bera. I residui **non sono perfettamente normali**, ma sono molto vicini. La forma più appuntita (leptocurtica) è comune nei dati reali e non invalida il modello per scopi di previsione.

3. Normal Q-Q Plot

Questo grafico confronta i quantili dei residui (punti blu) con i quantili di una distribuzione normale teorica (linea rossa).

- La stragrande maggioranza dei punti si allinea quasi perfettamente lungo la linea rossa. Si notano solo leggere deviazioni alle estremità, che corrispondono agli outlier.
- Questo è un risultato eccellente e rafforza la conclusione precedente. Nonostante la non-normalità statistica, la distribuzione **si avvicina molto a una distribuzione normale** per la maggior parte dei valori.

4. Correlogram (ACF dei Residui)

Questo grafico mostra se gli errori del modello sono correlati tra loro a diversi lag.

- **Nessuna** delle barre di autocorrelazione (a eccezione del lag 0, che è sempre 1) **supera in modo significativo l'area di confidenza blu**.
- Questo conferma visivamente il test di Ljung-Box ($\text{Prob}(Q) > 0.05$): quindi **non c'è alcuna autocorrelazione residua significativa**. Il modello ha catturato tutta la dipendenza temporale (sia a breve termine sia stagionale) presente nei dati.

4.5 Addestramento del modello sulla serie differenziata

Si addestrerà il modello dividendo il dataset in 80% training e 20% testing, questa volta utilizzando la serie differenziata.

```
1 # Divisione in training e test set (80% training, 20% test)
2 train_size = int(len(crime_time_series) * 0.8)
3 train, test = crime_time_series[:train_size], crime_time_series[
    train_size:]
4
5 # Addestramento del modello SARIMA con i parametri trovati
6 model = SARIMAX(train['Total Incidents'],
7                 order=(1, 1, 1),
8                 seasonal_order=(2, 1, 0, 52),
9                 trend='n')
10 model_fit = model.fit(dispatch=False)
11
12 # Previsioni sul test set
13 forecast_object = model_fit.get_forecast(steps=len(test))
14 forecast_mean = forecast_object.predicted_mean
15 forecast_ci = forecast_object.conf_int()
16
17 # Calcolo degli errori
18 mse = mean_squared_error(test['Total Incidents'], forecast_mean)
19 mae = mean_absolute_error(test['Total Incidents'], forecast_mean)
20 rmse = np.sqrt(mse)
```

```
1 Mean Absolute Error (MAE): 76.28
2 Root Mean Squared Error (RMSE): 97.63
```

Listing 10: Valori degli Errori

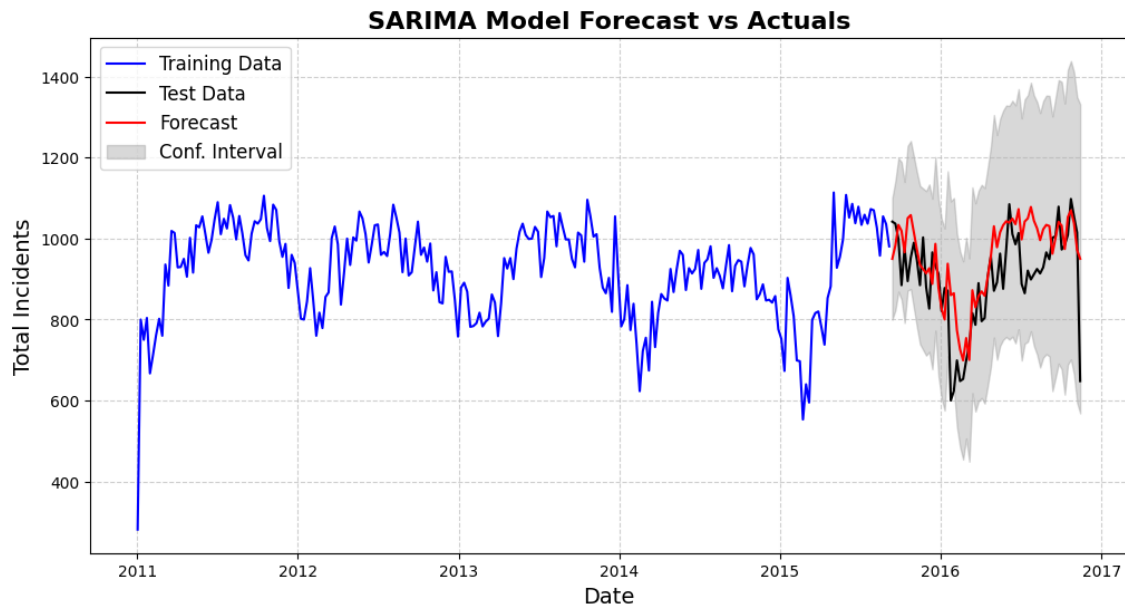


Figura 32: Modello SARIMAX serie differenziata

Analizzando il grafico del modello SARIMAX sulla serie differenziata (Figura 32), emergono differenze sostanziali rispetto al modello precedente:

- **Previsione (Linea rossa):** a differenza del modello sulla serie originale (che appariva "piatto"), qui la linea di previsione è **visivamente più dinamica e reattiva**, mostrando oscillazioni marcate. Tuttavia, a un'analisi attenta, si nota un evidente effetto di **ritardo di fase** (*lagging*): il modello tende a replicare i picchi e le valli della settimana precedente con un leggero ritardo, finendo per "inseguire" il dato reale piuttosto che anticiparlo.
- **Intervallo di confidenza:** rimane ampio, simile al modello precedente. Questo indica che, nonostante la stazionarietà sia stata raggiunta, l'incertezza intrinseca del rumore non è stata ridotta.

Le metriche d'errore (Listato 10), confrontate con il modello precedente, descrivono che:

- **MAE (Mean Absolute Error): 76.28.** Il valore è **praticamente identico** (lievemente peggiorato) rispetto al modello sulla serie originale (75.32). Questo conferma che la maggiore reattività visiva non si è tradotta in una maggiore precisione puntuale: l'errore commesso nel prevedere un picco in ritardo (quando il dato reale sta già scendendo) annulla i benefici della maggiore volatilità.

- **RMSE (Root Mean Squared Error): 97.63.** Si osserva un miglioramento marginale rispetto al precedente (100.55). La riduzione dell'RMSE suggerisce che il modello riesce a gestire leggermente meglio alcuni valori estremi, ma il divario con il MAE (circa 21) rimane significativo.
- **Conclusione:** questo risultato dimostra un concetto fondamentale nel forecasting: **la stazionarietà statistica è una condizione necessaria ma non sufficiente** per una buona previsione. Il modello differenziato ha corretto la struttura teorica dei dati, ma non disponendo di informazioni esterne, può solo basarsi sull'inerzia passata, risultando in una performance stagnante.

4.6 Miglioramento del modello SARIMAX: Feature Engineering e Variabili Esogene

Per potenziare la capacità predittiva del modello SARIMAX, sono state introdotte nuove **variabili esogene**. L'obiettivo è fornire al modello informazioni contestuali che vadano oltre il semplice andamento storico del numero di incidenti, consentendogli di interpretare in modo più accurato le fluttuazioni della serie temporale, come picchi stagionali o trend recenti.

Le feature sono state selezionate e costruite ponendo particolare attenzione al **ritardo temporale (lag)**. Per prevedere il numero di incidenti nella settimana t , vengono utilizzate esclusivamente informazioni disponibili fino alla settimana $t - 1$, garantendo così la validità del modello in uno scenario reale di previsione.

Le variabili esogene introdotte possono essere suddivise nelle seguenti categorie:

- **Variabili di composizione del crimine (lagged)**
 - *lag_firearm_rate*: percentuale di crimini con arma da fuoco registrata nella settimana precedente.
 - *lag_inside_rate*: percentuale di crimini avvenuti in ambienti chiusi (*Inside*) nella settimana precedente.

La composizione qualitativa del crimine può anticipare variazioni quantitative del numero totale di incidenti. Ad esempio, un aumento della violenza armata o uno spostamento verso crimini indoor, spesso legati a condizioni climatiche o dinamiche sociali, può fungere da segnale precoce di un cambiamento nel volume complessivo dei crimini. Poiché tali percentuali sono ottenute aggregando variabili binarie (0/1) a partire dal dataset grezzo, esse rappresentano una misura accurata della frequenza relativa dei fenomeni considerati.

- **Variabili di trend recente (rolling e lagged)**

- *lag_rolling_mean*: media mobile degli incidenti nelle ultime 4 settimane, traslata di una settimana.

Questa variabile cattura l'inerzia del fenomeno: se il trend dell'ultimo mese è in crescita o in diminuzione, è plausibile che la settimana successiva segua la stessa direzione. L'utilizzo di una media mobile consente inoltre di ridurre il rumore settimanale, fornendo al modello un segnale di trend più stabile.

- **Variabili di calendario (stagionalità)**

- *month*: mese dell'anno.
- *is_summer*: variabile binaria che indica i mesi estivi (giugno, luglio e agosto).

La criminalità presenta una marcata componente stagionale. I mesi estivi, caratterizzati da una maggiore attività all'aperto, flussi turistici e chiusura delle scuole, mostrano pattern differenti rispetto ai mesi invernali. L'inclusione di queste variabili consente al modello di modulare le previsioni in funzione del periodo dell'anno.

Per evitare errori di indicizzazione temporale e garantire la correttezza matematica delle feature, le percentuali (*rate*) sono state calcolate a partire dal dataset grezzo (*crime_data*). Le categorie di interesse sono state trasformate in flag numerici (0/1) e successivamente aggregate tramite media durante il ricampionamento settimanale.

```
1 # Copia del DataFrame originale
2 df = crime_time_series.copy()
3
4 # Crea flag numerici per le variabili categoriche
5 crime_data['is_firearm'] = (crime_data['Weapon'] == 'Firearm').astype(
    int)
6 crime_data['is_inside'] = (crime_data['Inside/Outside'] == 'Inside').
    astype(int)
7
8 # Aggregazione settimanale con le nuove variabili
9 crime_weekly = (
10     crime_data.set_index('DateTime')
11     .resample('W')
```



```

12     .agg({
13         'Total Incidents': 'sum', # Somma per il conteggio totale
14         'is_firearm': 'mean', # Media dei flag = Tasso (firearm_rate)
15         'is_inside': 'mean' # Media dei flag = Tasso (inside_rate)
16     })
17     .reset_index()
18 )
19
20 # Rinominazione delle colonne per coerenza
21 crime_weekly.rename(columns={
22     'is_firearm': 'firearm_rate',
23     'is_inside': 'inside_rate'
24 }, inplace=True)
25
26 # Rolling features (basate sui dati attuali)
27 crime_weekly['rolling_mean_4w'] = crime_weekly['Total Incidents'].
    rolling(window=4).mean()
28
29 # Variabili ritardate di 1 settimana
30 crime_weekly['lag_firearm_rate'] = crime_weekly['firearm_rate'].shift
    (1)
31 crime_weekly['lag_inside_rate'] = crime_weekly['inside_rate'].shift(1)
32 crime_weekly['lag_rolling_mean'] = crime_weekly['rolling_mean_4w'].
    shift(1)
33
34 # Variabili di calendario
35 crime_weekly['month'] = crime_weekly['DateTime'].dt.month
36 crime_weekly['is_summer'] = crime_weekly['month'].isin([6, 7, 8]).
    astype(int)
37
38 # Pulizia
39 # Le operazioni di shift creano dei NaN all'inizio (la prima settimana
    non ha una settimana precedente)
40 df_final = crime_weekly.dropna().copy()
41
42 # Definizione delle variabili esogene finali
43 exog_features = ['lag_firearm_rate', 'lag_inside_rate', '
    lag_rolling_mean', 'month', 'is_summer']
44
45 # --- Preparazione per SARIMAX ---

```

```

46 y = df_final['Total Incidents']
47 X = df_final[exog_features]

```

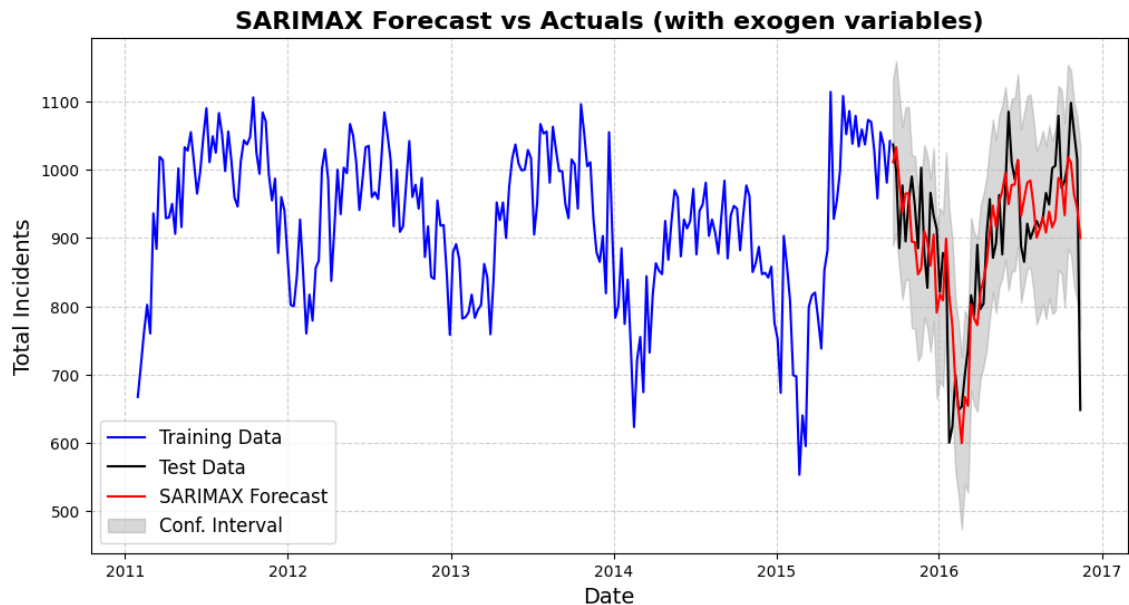


Figura 33: Modello SARIMAX migliorato

Le metriche di errore confermano che la risoluzione delle problematiche legate all'aggregazione delle feature ha portato a un modello **nettamente superiore** rispetto alla versione precedente basata sulla serie differenziata senza variabili esogene.

Metrica	SARIMAX (Precedente)	SARIMAX (Attuale)	Miglioramento
MAE	76.28	58.08	-23.9%
RMSE	97.62	76.96	-21.2%

- **Mean Absolute Error (MAE): 58.08.** L'errore medio si riduce di quasi il **24%**. Considerando un numero medio di circa 900 incidenti settimanali, questo valore corrisponde a uno scostamento percentuale di appena il **6.4%**, indicando un'elevata accuratezza predittiva.
- **Root Mean Squared Error (RMSE): 76.96.** La riduzione di oltre il **21%** dell'RMSE suggerisce che il modello è diventato più robusto anche nella gestione di picchi e anomalie, riducendo la frequenza di errori di grande entità.

Il confronto visivo tra i dati reali e le previsioni del modello evidenzia tre aspetti qualitativi di particolare rilievo:

1. **Cattura della volatilità:** a differenza del modello base, la versione attuale riesce a riprodurre efficacemente il brusco calo e la successiva rapida ripresa osservati all'inizio del 2016. Ciò indica che le variabili esogene trasmettono correttamente l'informazione relativa al trend di breve periodo.
2. **Intervallo di confidenza:** l'intervallo di confidenza rimane contenuto attorno alla previsione, segnalando una bassa incertezza del modello. La quasi totalità delle osservazioni reali ricade all'interno di tale fascia.
3. **Gestione delle anomalie finali:** nell'ultima parte del periodo osservato si nota un crollo verticale dei dati reali che il modello segue solo parzialmente. È plausibile che tale valore sia affetto da incompletezza dei dati; in questo contesto, la resistenza del modello a seguire il calo estremo rappresenta un segnale positivo di robustezza.

Il confronto grafico complessivo (Figura 33) rafforza ulteriormente le conclusioni emerse dall'analisi quantitativa, confermando l'efficacia dell'introduzione delle variabili esogene nel modello SARIMAX.

Plottando i residui, è possibile inoltre analizzarli per avere delle osservazioni aggiuntive.

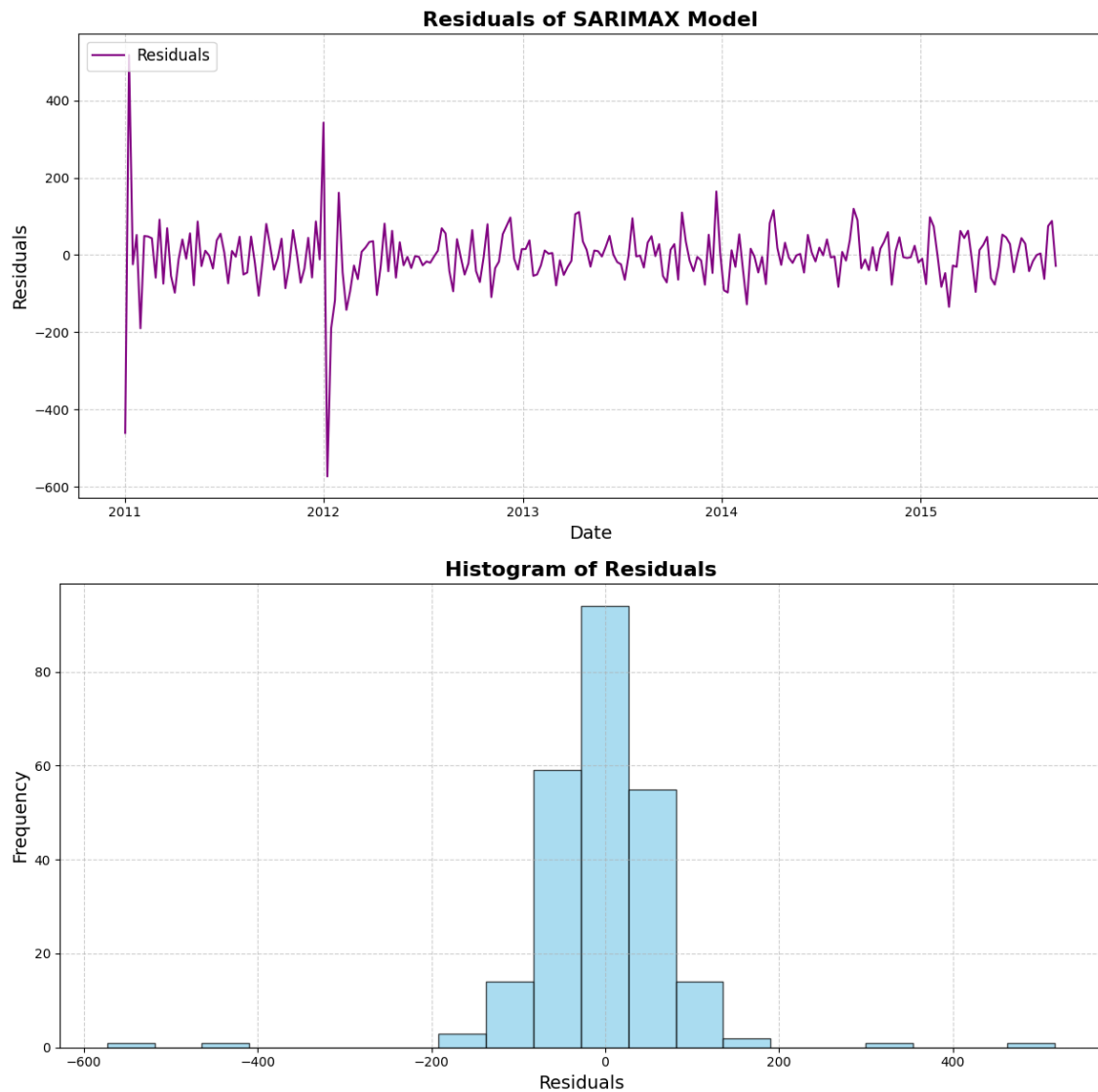


Figura 34: Grafici dei residui

Osservando i due grafici (Figura 34), si può notare che:

1. Grafico dei residui

Dall'analisi del grafico emergono tre osservazioni positive:

- **Media Attorno a Zero:** I residui oscillano costantemente attorno al valore zero, indicando che il modello non soffre di un *bias* sistematico (non tende né a sovrastimare né a sottostimare).
- **Varianza Costante (Omoschedasticità):** L'ampiezza delle oscillazioni è ragionevolmente stabile per tutto il periodo, suggerendo che la varianza degli errori sia costante.

- **Assenza di Pattern:** Non sono visibili cicli o schemi prevedibili. Gli errori appaiono casuali, confermando che il modello ha catturato con successo la struttura temporale e l'autocorrelazione presenti nei dati.

2. Istogramma dei residui

L'istogramma conferma ulteriormente la bontà del modello:

- **Forma a Campana:** La distribuzione dei residui ha una chiara e simmetrica forma a campana, molto simile a una distribuzione normale.
- **Centratura sullo Zero:** Il picco della distribuzione è centrato esattamente sullo zero, rafforzando l'idea che l'errore medio del modello sia nullo.

4.7 Conclusioni ed osservazioni

L'analisi ha mostrato che l'utilizzo di modelli basati esclusivamente sull'andamento storico della serie temporale consente di catturare il trend medio del fenomeno, ma presenta limiti significativi nella gestione della variabilità e dei cambiamenti improvvisi. Le trasformazioni della serie, come la differenziazione stagionale, migliorano parzialmente la dinamica delle previsioni, senza tuttavia garantire un effettivo aumento della capacità predittiva.

L'introduzione di variabili esogene ha rappresentato un miglioramento sostanziale. L'integrazione di informazioni contestuali e di indicatori di breve periodo ha permesso al modello SARIMAX di ridurre significativamente gli errori di previsione e di ottenere risultati più stabili e coerenti con l'evoluzione reale dei dati.